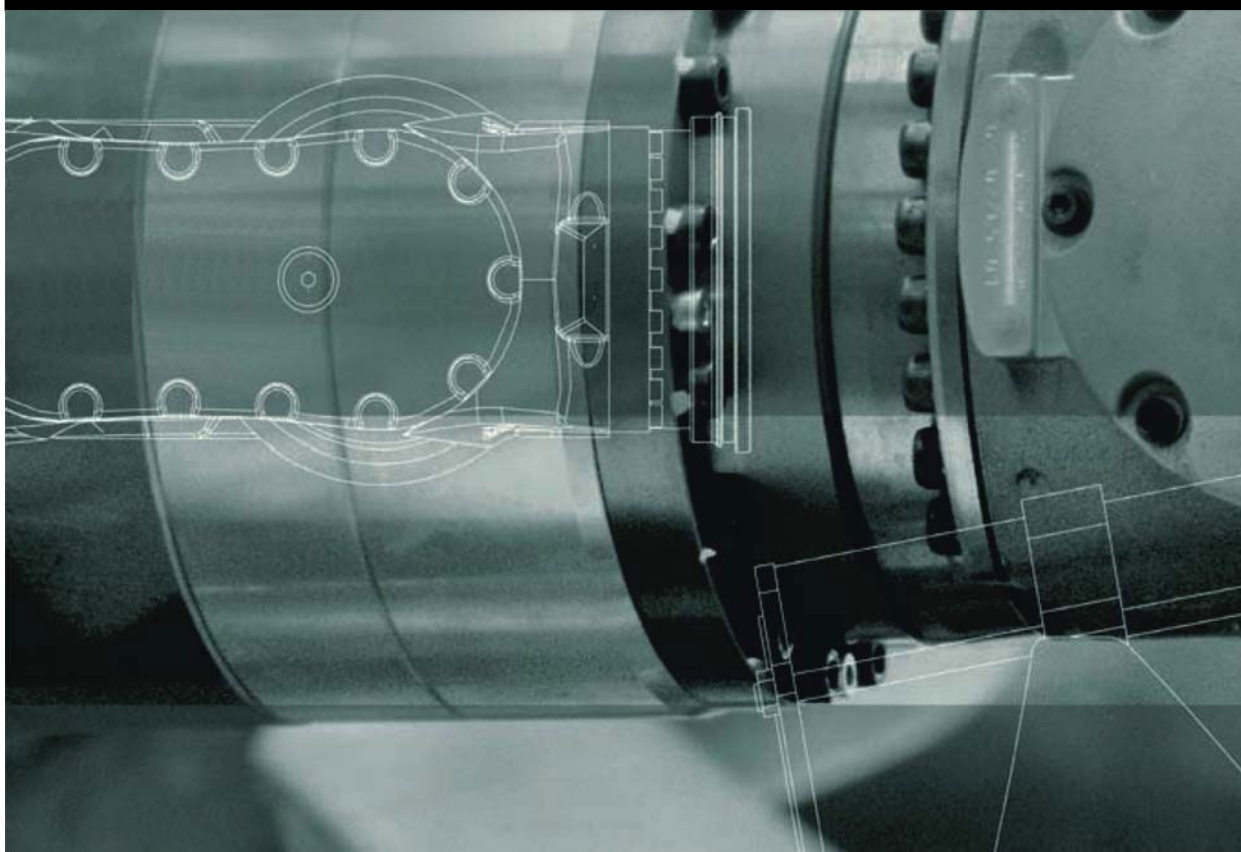


KUKA System Technology

KUKA Roboter GmbH

KUKA.Ethernet KRL 3.0

Pour logiciel KUKA System Software 8.5



Edition: 30.03.2017

Version: KST Ethernet KRL 3.0 V3

© Copyright 2017

KUKA Roboter GmbH
Zugspitzstraße 140
D-86165 Augsburg
Allemagne

La présente documentation ne pourra être reproduite ou communiquée à des tiers, même par extraits, sans l'autorisation expresse du KUKA Roboter GmbH.

Certaines fonctions qui ne sont pas décrites dans la présente documentation peuvent également tourner sur ce contrôleur. Dans ce cas, l'utilisateur ne pourra exiger ces fonctions en cas de nouvelle livraison ou de service après-vente.

Nous avons vérifié la concordance entre cette brochure et le matériel ainsi que le logiciel décrits. Des différences ne peuvent être exclues. Pour cette raison, nous ne pouvons garantir la concordance exacte. Les informations de cette brochure sont néanmoins vérifiées régulièrement afin d'inclure les corrections indispensables dans l'édition suivante.

Sous réserve de modifications techniques n'influençant pas les fonctions.

Traduction de la documentation originale

KIM-PS5-DOC

Publication:	Pub KST Ethernet KRL 3.0 (PDF) fr
Structure de livre:	KST Ethernet KRL 3.0 V3.1
Version:	KST Ethernet KRL 3.0 V3

Table des matières

1	Introduction	7
1.1	Cible	7
1.2	Documentation du robot industriel	7
1.3	Représentation des remarques	7
1.4	Termes utilisés	8
1.5	Marques	9
1.6	Licences	9
2	Description du produit	11
2.1	Aperçu d'Ethernet KRL	11
2.2	Configuration d'une connexion Ethernet	11
2.2.1	Comportement en cas de perte de connexion	11
2.2.2	Surveillance d'une connexion	12
2.3	Échange de données	12
2.4	Sauvegarde de données	13
2.5	Mode client-server	14
2.6	Types de protocole	14
2.7	Messages d'événements	15
2.8	Traitement de défauts	15
2.9	Utilisation conforme aux fins prévues	16
3	Sécurité	17
4	Installation	19
4.1	Conditions requises par le système	19
4.2	Installation avec WorkVisual	19
4.2.1	Installation ou mise à jour d'Ethernet KRL	19
4.2.2	Désinstallation d'Ethernet KRL	19
4.3	Installation avec smarthMI	20
4.3.1	Installation ou mise à jour d'Ethernet KRL	20
4.3.2	Désinstallation d'Ethernet KRL	21
5	Configuration	23
5.1	Connexion réseau via la KLI du contrôleur du robot	23
6	Programmation	25
6.1	Configuration d'une connexion Ethernet	25
6.1.1	Structure XML des propriétés de connexion	25
6.1.2	Structure XML de la réception de données	28
6.1.3	Structure XML pour l'envoi de données	31
6.1.4	Configuration selon le schéma XPath	31
6.2	Fonctions pour l'échange de données	32
6.2.1	Astuces de programmation	33
6.2.2	Initialiser et supprimer une connexion	34
6.2.3	Ouvrir et fermer une connexion	36
6.2.4	Envoi de données	36
6.2.5	Lecture de données	39
6.2.6	Suppression de mémoires de données	41

6.2.7	EKI_STATUS – Structure pour valeurs renvoyées spécifiques à la fonction	42
6.2.8	Configuration de messages d'évènements	43
6.2.9	Réception de blocs de données XML complets	44
6.2.10	Édition de blocs de données incomplets	44
6.2.11	EKI_CHECK() – Vérifier que les fonctions sont exemptes de défauts	44
7	Configurations et de programmes à titre d'exemple	47
7.1	Intégration d'un programme de serveur et d'exemples	47
7.2	Interface utilisateur du programme de serveur	48
7.2.1	Réglage des paramètres de communication dans le programme de serveur	49
7.3	Exemple « BinaryFixed »	50
7.4	Exemple « BinaryStream »	52
7.5	Exemple « XmlTransmit »	53
7.6	Exemple « XmlServer »	54
7.7	Exemple « XmlCallback »	55
8	Diagnostic	59
8.1	Affichage des données de diagnostic	59
9	Messages	61
9.1	Protocole de défauts (table de messages EKI)	61
9.2	Informations concernant les messages	61
9.3	Messages de système du module: EthernetKRL (EKI)	61
9.3.1	EKI00002	61
9.3.2	EKI00003	63
9.3.3	EKI00006	64
9.3.4	EKI00007	65
9.3.5	EKI00009	66
9.3.6	EKI00010	67
9.3.7	EKI00011	68
9.3.8	EKI00012	70
9.3.9	EKI00013	71
9.3.10	EKI00014	72
9.3.11	EKI00015	74
9.3.12	EKI00016	75
9.3.13	EKI00017	78
9.3.14	EKI00018	82
9.3.15	EKI00019	83
9.3.16	EKI00020	85
9.3.17	EKI00021	88
9.3.18	EKI00022	89
9.3.19	EKI00023	90
9.3.20	EKI00024	91
9.3.21	EKI00027	93
9.3.22	EKI00512	93
9.3.23	EKI00768	95
9.3.24	EKI01024	95
9.3.25	EKI01280	96
9.3.26	EKI01536	99

9.3.27	EKI01792	99
9.3.28	EKI02048	100
10	Annexe	103
10.1	Structure XML avancée pour les propriétés de connexion	103
10.2	Augmentation de la mémoire	103
10.3	Désactivation de l'émission de messages et du protocole de messages	104
10.4	Référence d'instruction	104
10.4.1	Initialiser, ouvrir, fermer et effacer une connexion	104
10.4.2	Envoyer des données	105
10.4.3	Écrire des données	107
10.4.4	Lire des données	108
10.4.5	Vérifier que la fonction est exempte de défauts	112
10.4.6	Effacer, verrouiller, déverrouiller et contrôler la mémoire de données	112
11	SAV KUKA	115
11.1	Demande d'assistance	115
11.2	Assistance client KUKA	115
	Index	123

1 Introduction

1.1 Cible

Cette documentation s'adresse à l'utilisateur avec les connaissances suivantes :

- Connaissances approfondies de la programmation KRL
- Connaissances approfondies du système de la commande de robot
- Connaissances approfondies XML
- Connaissances approfondies de réseau



Pour une application optimale de nos produits, nous recommandons à nos clients une formation au KUKA College. Consultez notre site Internet www.kuka.com ou adressez-vous à une de nos filiales pour tout complément d'information sur notre programme de formation.

1.2 Documentation du robot industriel

La documentation du robot industriel est formée des parties suivantes :

- Documentation pour l'ensemble mécanique du robot
- Documentation pour le contrôleur de robot
- Manuels de service et de programmation pour le logiciel système
- Instructions relatives aux options et accessoires
- Catalogue des pièces sur support de données

Chaque manuel est un document individuel.

1.3 Représentation des remarques

Sécurité

Ces remarques se réfèrent à la sécurité et **doivent** donc être respectées impérativement.



DANGER

Ces remarques signifient que des blessures graves, voire même mortelles vont sûrement ou très vraisemblablement **être** la conséquence de l'absence de mesures de précaution.



AVERTISSEMENT

Ces remarques signifient que des blessures graves, voire même mortelles **peuvent être** la conséquence de l'absence de mesures de précaution.



ATTENTION

Ces remarques signifient que des blessures légères **peuvent être** la conséquence de l'absence de mesures de précaution.

AVIS

Ces remarques signifient que des dommages matériels **peuvent être** la conséquence de l'absence de mesures de précaution.



Ces remarques renvoient à des informations importantes pour la sécurité ou à des mesures de sécurité générales.
Ces remarques ne se réfèrent pas à des dangers isolés ou à des mesures de sécurité individuelles.

Cette remarque attire l'attention sur des procédures permettant d'éviter ou d'éliminer des cas d'urgence ou de panne :

**INSTRUCTIONS
DE SÉCURITÉ**

Respecter strictement la procédure suivante !

Les procédures caractérisées par cette remarque **doivent** être respectées avec précision.

Remarques

Ces remarques facilitent le travail ou renvoient à des informations supplémentaires.



Remarque facilitant le travail ou renvoi à des informations supplémentaires.

1.4 Termes utilisés

Terme	Description
Flux de données	Suivi continu de blocs de données dont la fin n'est pas prévisible. Les blocs de données individuels sont de type facultatif mais fixe. La quantité de blocs de données par unité temporelle (transmission de données) peut varier. Seul un accès séquentiel aux données est possible.
EKI	Ethernet KRL Interface
EOS	End Of Stream (séquence de suffixes) Séquence de suffixes désignant la fin d'un bloc de données
Ethernet	Ethernet est une technologie de réseau de données pour réseaux de données locaux (LAN). Elle permet d'échanger des données sous forme de cadres de données entre les participants raccordés.
FIFO	Procédé avec lequel une mémoire de données peut être traitée ■ First In First Out: Les éléments sauvegardés en premier sont retirés en premier de la mémoire. ■ Last In First Out: les éléments sauvegardés en dernier sont retirés en premier de la mémoire.
LIFO	
KLI	KUKA Line Interface Bus de ligne permettant d'intégrer l'installation dans le réseau du client
KR C	KUKA Roboter Controller KR C correspond à la commande de robot KUKA
KRL	KUKA Robot Language KRL est le langage de programmation de robot KUKA.
smarHMI	smart Human-Machine Interface KUKA smarHMI correspond à l'interface utilisateur du logiciel de KUKA System.
Socket	Interface logicielle reliant les adresses IP et les numéros de port entre eux.

Terme	Description
TCP/IP	Transmission-Control-Protocol Protocole de l'échange de données entre les participants d'un réseau. TCP crée un canal virtuel entre 2 points finaux d'une connexion réseau. Des données peuvent être transmises dans les deux sens sur ce canal.
UDP/IP	User Datagram Protocol Protocole sans connexion relatif à la transmission des données entre les usagers d'un réseau.
IP	Internet Protocol Le protocole internet a pour tâche de définir les sous-réseaux par le biais d'adresses MAC physiques.
XML	Extensible Markup Language Standard pour la création de documents lisibles pour les machines et les hommes sous forme d'arborescence prédéfinie.
XPath	XML Path Language Langage pour écrire et lire des parties d'un document XML

1.5 Marques

.NET Framework est une marque de la Microsoft Corporation.

Windows est une marque déposée par Microsoft Corporation.

1.6 Licences

Les termes de la licence KUKA et les termes de licence du logiciel Open-Source utilisé se trouvent dans les dossiers suivants :

- Sur le support de données contenant les fichiers d'installation du logiciel KUKA sous .LICENSE
- Après l'installation sur le contrôleur de robot, sous D:\KUKA_OPT\nom de pack d'options\LICENSE
- Après l'installation dans WorkVisual dans la fenêtre **Catalogues** de l'onglet **Options**, dans le dossier de licence, sous le nom du pack d'options



Des informations supplémentaires concernant les licences Open-Source peuvent être demandées à : opensource@kuka.com.

2 Description du produit

2.1 Aperçu d'Ethernet KRL

Fonctions	Ethernet KRL est un pack d'options rechargeable disposant des fonctions suivantes : ■ Échange de données via l'EKI ■ Réception de données XML d'un système externe ■ Envoi de données XML à un système externe ■ Réception de données binaires d'un système externe ■ Envoi de données binaires à un système externe
Propriétés	■ Contrôleur de robot et système externe en tant que client ou serveur ■ Configuration de connexions avec un fichier de configuration basé sur XML ■ Configuration de « Messages d'évènements » ■ Surveillance de connexions avec un Ping sur le système externe ■ Lecture et écriture de données de l'interpréteur Submit ■ Lecture et écriture de données de l'interpréteur robot
Communication	Les données sont transmises via le protocole TCP/IP. L'utilisation du protocole UDP/IP est possible mais n'est pas recommandée (protocole réseau sans connexion, p. ex. pas de détection de perte de données).

2.2 Configuration d'une connexion Ethernet

La connexion Ethernet est configurée avec un fichier XML. Un fichier de configuration doit être défini dans le répertoire C:\KRC\ROBOTER\Config\User\Common\EthernetKRL du contrôleur de robot pour chaque connexion. La configuration est lue lors de l'initialisation d'une connexion.

Les connexions Ethernet peuvent être créées et contrôlées par l'interpréteur robot ou Submit. Les canaux peuvent être utilisés interconnexion, c'est-à-dire qu'un canal ouvert dans l'interpréteur Submit peut également être utilisé par l'interpréteur robot.

La suppression d'une connexion peut être reliée à des actions d'interpréteur robot et Submit ou des actions de système.

Lors d'une utilisation en interconnexion, certaines directives doivent être respectées afin d'éviter un comportement inattendu du programme.

(>>> 6.2.1 "Astuces de programmation" Page 33)

2.2.1 Comportement en cas de perte de connexion

Les caractéristiques et fonctions de l'EKI suivantes garantissent que les données reçues puissent être traitées de façon fiable :

- Lorsque la limite d'une mémoire de données est atteinte, une connexion est automatiquement coupée.
- Si un défaut apparaît lors de la réception de données, une connexion est automatiquement coupée.
- Lorsque la connexion est coupée, les mémoires de données continuent à être lues.
- En cas de perte de connexion, il est possible de la rétablir sans influencer les données sauvegardées.

- Une perte de connexion peut être affichée, p. ex. avec un drapeau.
- Le message de défaut concernant le défaut ayant provoqué la perte de connexion peut être affiché sur la smartHMI.

2.2.2 Surveillance d'une connexion

Une connexion peut être surveillée avec un Ping sur le système externe. (élément <ALIVE.../> dans la configuration de connexion)

Lorsque la connexion est établie avec succès, un drapeau ou une sortie peuvent être activés, en fonction de la configuration. Tant que le Ping est envoyé régulièrement et que la connexion avec le système externe est active, la sortie ou le drapeau sont activés. Dès que la connexion avec le système externe est interrompue, la sortie ou le drapeau sont effacés.

2.3 Échange de données

Aperçu

Avec KUKA.Ethernet KRL, le contrôleur de robot peut recevoir des données d'un système externe ou bien envoyer des données à un système externe.

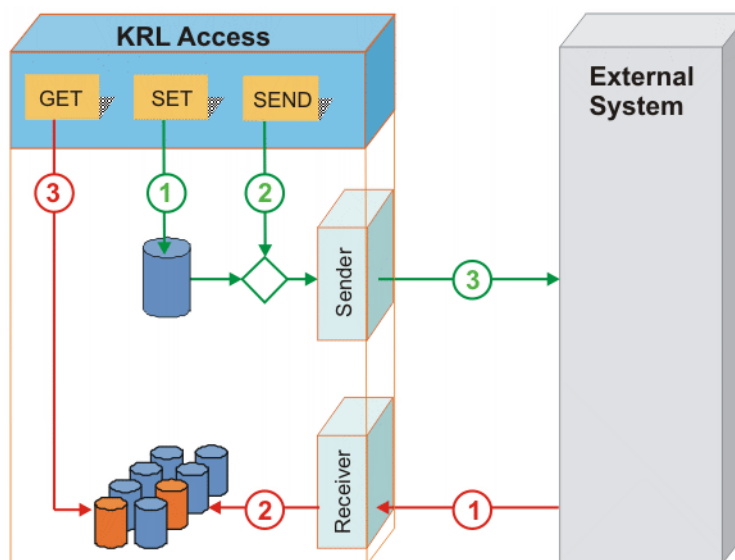


Fig. 2-1: Aperçu du système

Réception de données

Déroulement principal (en rouge) (>>> Fig. 2-1) :

1. Le système externe envoie des données transmises par un protocole et reçues par l'EKI.
2. Les données sont sauvegardées dans une mémoire de données de façon structurée.
3. L'accès est effectué de façon structurée à partir d'un programme KRL. Les données sont lues et copiées dans des variables KRL à l'aide d'instructions KRL.

Envoi de données

Déroulement principal (en vert) (>>> Fig. 2-1) :

1. Les données sont écrites dans une mémoire de données de façon structurée à l'aide d'instructions KRL.
2. Les données sont lues dans la mémoire de données avec une instruction KRL.
3. EKI envoie les données au système externe avec un protocole.



Il est possible d'envoyer directement des données sans les avoir sauvegardées auparavant dans une mémoire.

2.4 Sauvegarde de données

Description Toutes les données reçues sont automatiquement sauvegardées et sont donc disponibles en KRL. Lors de la sauvegarde, les données XML et binaires sont traitées différemment.

Chaque mémoire de données est réalisée en tant que pile. Les mémoires individuelles sont lues en mode FIFO ou LIFO.

Données XML Les données reçues sont extraites et sauvegardées dans différentes mémoires en fonction de leur type (une mémoire par valeur).

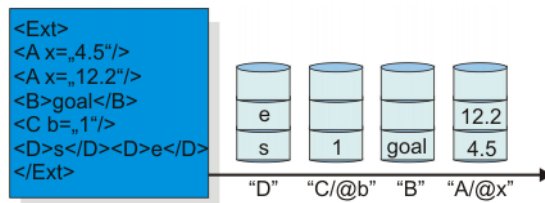


Fig. 2-2: Mémoire de données XML

Données binaires Les données reçues ne sont pas extraites ni interprétées. Une seule mémoire existe pour une connexion en mode binaire.

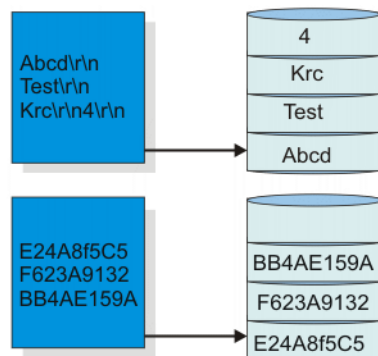


Fig. 2-3: Mémoire de données binaire

Processus de lecture

Les éléments de données sont retirés de la mémoire dans le même ordre que celui avec lequel ils y ont été sauvegardés (FIFO). Il est possible de configurer le procédé inverse, avec lequel l'élément de données sauvegardé en dernier dans la mémoire est retiré le premier (LIFO).

Chaque mémoire se voit affecter une limite commune de nombre maximum de données pouvant être sauvegardées. Si la limite est dépassée, la connexion Ethernet est immédiatement coupée afin d'éviter la réception de données supplémentaires. Les données actuellement reçues sont encore sauvegardées et la mémoire est augmentée de « 1 ». Les mémoires peuvent continuer à être traitées. La connexion peut être rétablie avec la fonction EKI_OPEN().

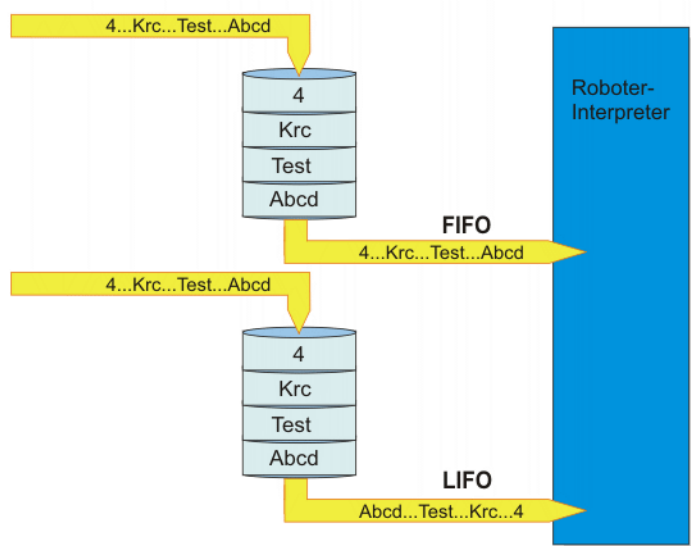


Fig. 2-4: Aperçu du processus de lecture

2.5 Mode client-server

Description

Le contrôleur de robot et le système se connectent en tant que client et serveur. Ceci faisant, le système externe peut être client ou serveur. Le nombre de connexions actives est limité à 16.

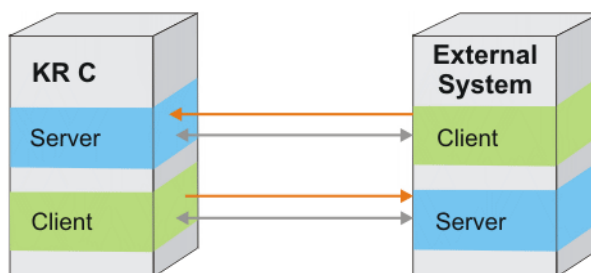


Fig. 2-5: Mode client - serveur

Lorsque l'EKI est configuré en tant que serveur, un seul client peut se connecter avec le serveur. Si plusieurs connexions sont nécessitées, il faudra également créer plusieurs serveurs EKI. Il est possible d'utiliser plusieurs clients et serveurs simultanément dans l'EKI.

2.6 Types de protocole

Description

Les données transmises peuvent être comprimées sous différents formats.

Les formats suivants sont autorisés :

- Structure XML pouvant être configurée à souhait
- Bloc de données binaires avec longueur fixe
- Bloc de données binaires variables avec séquence de suffixes

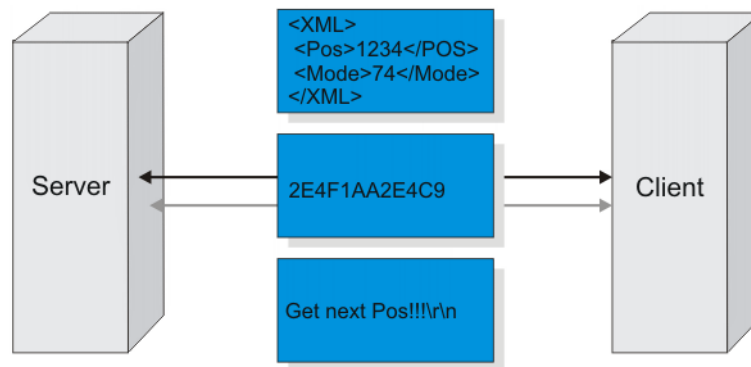


Fig. 2-6: Types de protocole

Les deux formats de données binaires ne peuvent pas être utilisés simultanément sur une connexion.

Les combinaisons suivantes sont possibles :

Connexion Vx	V1	V2	V3	V4	V5
Binaire fixe	✓	✗	✓	✗	✗
Binaire variable	✗	✗	✗	✓	✓
XML	✓	✓	✗	✗	✓

Exemples

F F E A 1 6 C C 0 1 2 3 B E 9 7 8 F F F

Fig. 2-7: Données binaires avec longueur fixe (10 octets)

H E L L O : E N D
G e t n e x t P o s ! ! ! \ R \ N

Fig. 2-8: Données binaires variables avec séquence de suffixes

2.7 Messages d'évènements

L'activation d'une sortie ou d'un drapeau permet de signaler les évènements suivants :

- La connexion est active.
- Un élément XML individuel est arrivé.
- Une structure XML complète ou un bloc de données binaires complet sont arrivés.

(>>> 6.2.8 "Configuration de messages d'évènements" Page 43)

2.8 Traitement de défauts

Description

KUKA.Ethernet KRL met des fonctions à disposition pour l'échange de données entre le contrôleur de robot et le système externe.

Chacune de ces fonctions renvoie des valeurs. Ces valeurs de retour peuvent être interrogées et évaluées dans le programme KRL.

Les valeurs suivantes sont renvoyées selon la fonction :

- Numéro de défaut
- Nombre d'éléments se trouvant encore dans la mémoire après l'accès.

- Nombre d'éléments lus dans la mémoire
- Information indiquant s'il y a une connexion
- Horodatage de l'élément de données pris dans la mémoire

(>>> 6.2.7 "EKI_STATUS – Structure pour valeurs renvoyées spécifiques à la fonction" Page 42)

Un message est émis sur la smartHMI et dans la table de messages EKI pour chaque défaut. L'émission automatique de messages peut être désactivée.

2.9 Utilisation conforme aux fins prévues

Utilisation

Le pack d'options KUKA.Ethernet KRL est prévu pour l'échange de données entre le contrôleur de robot et un système externe via Ethernet pendant la durée de traitement de programme de l'interpréteur Submit ou robot.

Utilisation non conforme

Toutes les utilisations divergentes des fins prévues sont considérées comme non conformes et sont interdites. Dans ce cas, le fabricant décline expressément toute responsabilité pour les dommages éventuels occasionnés. Le risque est à la seule charge de l'exploitant.

Exemples d'utilisations non conformes :

- La communication via Ethernet ne remplace pas un système de bus de champ et n'est pas prévue pour la commande de l'interface Automatique Externe.
- KUKA.Ethernet KRL ne garantit aucun comportement temporel déterministe, c'est-à-dire qu'on ne peut pas partir du principe que des données appelées soient disponibles avec une temporisation constante.
- KUKA.Ethernet KRL n'est pas approprié pour établir une communication cyclique en temps réel du contrôleur de robot.

3 Sécurité

Cette documentation contient des remarques relatives à la sécurité se référant de façon spécifique au logiciel décrit ici.

Les informations fondamentales relatives à la sécurité concernant le robot industriel peuvent être consultées dans le chapitre « Sécurité » du manuel de service et de programmation pour intégrateurs de systèmes ou dans le manuel de service et de programmation pour l'utilisateur final.



Il faut respecter le chapitre "Sécurité" du manuel de service et de programmation du logiciel System Software. Un danger de mort, un risque de blessures graves ou de dommages matériels importants pourraient sinon s'ensuivre.

4 Installation

Le pack d'options peut être installé sur le contrôleur de robot soit via la smarHMI ou bien via WorkVisual.

4.1 Conditions requises par le système

Matériel	■ KR C4
	■ Système externe
Logiciels	■ KUKA System Software 8.5
	■ WorkVisual 5.0

4.2 Installation avec WorkVisual

4.2.1 Installation ou mise à jour d'Ethernet KRL

Description Le pack d'options est installé dans WorkVisual et ajouté au projet. Lors du transfert du projet, le pack d'options est installé automatiquement sur le contrôleur de robot.

 Il est conseillé d'archiver toutes les données correspondantes avant la mise à jour d'un logiciel.

Condition préalable

- Groupe d'utilisateurs Expert
- Mode T1 ou T2.
- Aucun programme n'est sélectionné.
- Connexion réseau entre le PC et le contrôleur de robot
- Le pack d'options est disponible sous forme de fichier KOP.

Procédure

1. Installer le pack d'options **EthernetKRL** dans WorkVisual.
2. Charger le projet actif du contrôleur de robot.
3. Insérer le pack d'options **EthernetKRL** dans le projet.
4. Transférer le projet de WorkVisual sur le contrôleur de robot et l'activer.
5. La smarHMI affiche la question de sécurité *Voulez-vous autoriser l'activation du projet [...] ?*. Le projet actif s'écrase lors de l'activation. Lorsqu'aucun projet pertinent n'est écrasé : Confirmer la question avec **Oui**.
6. Une vue d'ensemble avec les modifications et une question de sécurité s'affichent sur la smarHMI. Répondre par **Oui**. Le pack d'options est installé et le contrôleur de robot procède à un redémarrage.

 Pour tout complément d'informations concernant les séquences d'opération à effectuer dans WorkVisual, veuillez consulter la documentation de WorkVisual.

Fichier LOG Un fichier de protocole LOG est créé sous C:\KRC\ROBOTER\LOG.

4.2.2 Désinstallation d'Ethernet KRL

 Il est conseillé d'archiver toutes les données correspondantes avant la désinstallation d'un logiciel.

Condition préalable

- Groupe d'utilisateurs Expert
- Mode T1 ou T2.
- Aucun programme n'est sélectionné.
- Connexion réseau entre le PC et le contrôleur de robot

Procédure

1. Charger le projet du contrôleur de robot.
2. Supprimer le pack d'options **EthernetKRL** du projet. Une fenêtre avec les modifications est affichée.
3. Transférer le projet de WorkVisual sur le contrôleur de robot et l'activer.
4. Confirmer la question de sécurité *Voulez-vous autoriser l'activation du projet [...] ?* par **Oui** sur la smartHMI.
5. Une vue d'ensemble avec les modifications et une question de sécurité s'affichent sur la smartHMI. Répondre par **Oui**. Le pack d'options est désinstallé et le contrôleur de robot procède à un redémarrage.



Pour tout complément d'informations concernant les séquences d'opération à effectuer dans WorkVisual, veuillez consulter la documentation de WorkVisual.

Fichier LOG

Un fichier de protocole LOG est créé sous C:\KRC\ROBOTER\LOG.

4.3 Installation avec smartHMI**4.3.1 Installation ou mise à jour d'Ethernet KRL**

Il est conseillé d'archiver toutes les données correspondantes avant la mise à jour d'un logiciel.

Condition préalable

- Droits des utilisateurs : groupe de fonctions **Configuration générale**
- Mode T1 ou T2.
- Aucun programme n'est sélectionné.
- Clé USB avec le pack d'options (fichier KOP)



Il est recommandé d'utiliser une clé USB KUKA. Si une clé d'un autre fabricant est utilisée, des données peuvent être perdues.

Procédure

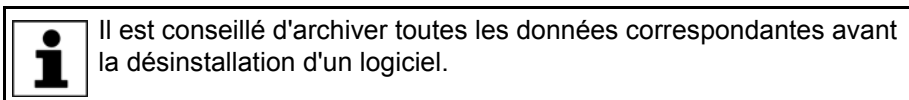
1. Connecter la clé USB au contrôleur du robot ou au smartPAD.
2. Dans le menu principal, sélectionner **Mise en service > Logiciel supplémentaire**.
3. Appuyer sur **Nouveau logiciel** : dans la colonne **Nom**, l'option **EthernetKRL** doit être affichée et dans la colonne **Chemin d'accès**, l'unité **E:** ou **K:**.
Si ce n'est pas le cas, appuyer sur **Actualiser**.
4. Lorsque les options nommées sont affichées, continuer avec l'opération 5.
Si ce n'est pas le cas, il faudra tout d'abord procéder à la configuration du chemin à partir duquel l'installation sera effectuée :
 - a. Appuyer sur le bouton **Configurer**.
 - b. Marquer une ligne dans la zone **Chemins d'installation pour options**.

Remarque : si la ligne contient déjà un chemin d'accès, celui-ci sera écrasé.

- c. Appuyer sur **Sélection de dossier**. Les unités existantes sont affichées.
- d. Si la clé est connectée au contrôleur du robot : Marquer **E:**
Si la clé est connectée au smartPAD : **K:** au lieu de **E:**
- e. Appuyer sur **Sauvegarder**. La zone **Chemins d'installation pour options** est à nouveau affichée. Elle contient le nouveau chemin.
- f. Marquer la ligne avec le nouveau chemin et appuyer encore une fois sur **Sauvegarder**.
5. Cocher **EthernetKRL** et appuyer sur **Installer**. Confirmer la remarque d'installation par **OK**.
6. La question de sécurité *Voulez-vous autoriser l'activation du projet [...] ?* s'affiche. Le projet actif s'écrase lors de l'activation. Lorsqu'aucun projet pertinent n'est écrasé : Confirmer la question avec **Oui**.
7. Une vue d'ensemble avec les modifications et une question de sécurité s'affiche. Répondre par **Oui**. Le pack d'options est installé et le contrôleur de robot procède à un redémarrage.
8. Retirer la clé.

Fichier LOG Un fichier de protocole LOG est créé sous C:\KRC\ROBOTER\LOG.

4.3.2 Désinstallation d'Ethernet KRL



Condition préalable

- Droits des utilisateurs : groupe de fonctions **Configuration générale**
- Mode T1 ou T2.
- Aucun programme n'est sélectionné.

Procédure

1. Dans le menu principal, sélectionner **Mise en service > Logiciel supplémentaire**.
2. Cocher **EthernetKRL** et appuyer sur **Désinstaller**. Répondre à la question de sécurité par **Oui**.
3. Confirmer la question de sécurité *Voulez-vous autoriser l'activation du projet [...] ?* par **Oui**.
4. Une vue d'ensemble avec les modifications et une question de sécurité s'affiche. Répondre par **Oui**. Le pack d'options est désinstallé et le contrôleur de robot procède à un redémarrage.

Fichier LOG Un fichier de protocole LOG est créé sous C:\KRC\ROBOTER\LOG.

5 Configuration

5.1 Connexion réseau via la KLI du contrôleur du robot

Une connexion réseau via la KLI de la commande du robot doit être établie pour l'échange de données via Ethernet.



Des informations supplémentaires concernant la configuration de réseau KLI sont fournies dans le manuel de service et de programmation du logiciel System Software.

En fonction de la spécification, les interfaces Ethernet suivantes sont disponibles en option à l'interface client de la commande du robot :

- Interface X66 (1 emplacement)
- Interface X67.1-3 (3 emplacements)



Pour tout complément d'information concernant les interfaces Ethernet, veuillez consulter le manuel ou les instructions de montage du contrôleur de robot.

6 Programmation

6.1 Configuration d'une connexion Ethernet

Aperçu

Une connexion Ethernet est configurée avec un fichier XML. Un fichier de configuration doit être défini dans le répertoire C:\KRC\ROBOTER\Config\User\Common\EthernetKRL du contrôleur de robot pour chaque connexion.



Les fichiers XML sont contextuels. Il faut respecter les majuscules / minuscules.

Le nom du fichier XML est simultanément la clé d'accès en KRL.

Exemple : ... \EXT.XML → EKI_INIT("EXT")

```
<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL></EXTERNAL>
    <INTERNAL></INTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <ELEMENTS></ELEMENTS>
  </RECEIVE>
  <SEND>
    <ELEMENTS></ELEMENTS>
  </SEND>
</ETHERNETKRL>
```

Paragraphe	Description
<CONFIGURATION> ... </CONFIGURATION>	Configuration des paramètres de connexion entre le système externe et l'EKI (>>> 6.1.1 "Structure XML des propriétés de connexion" Page 25)
<RECEIVE> ... </RECEIVE>	Configuration de la structure de réception reçue par le contrôleur de robot (>>> 6.1.2 "Structure XML de la réception de données" Page 28)
<SEND> ... </SEND>	Configuration de la structure d'envoi envoyée par le contrôleur de robot (>>> 6.1.3 "Structure XML pour l'envoi de données" Page 31)

6.1.1 Structure XML des propriétés de connexion

Description

Les réglages pour le système externe sont définis dans le paragraphe <EXTERNAL> ... </EXTERNAL> :

Elément	Description
TYPE	Définit si le système externe communique avec le contrôleur de robot en tant que serveur ou en tant que client (option) ■ Serveur : le système externe est un serveur. ■ Client : le système externe est un client. Valeur par défaut : serveur
IP	Adresse IP du système externe lorsque celui-ci est défini en tant que serveur (TYPE = serveur). Lorsque TYPE = client, l'adresse IP est ignorée.
PORT	Numéro de port du système externe lorsque celui-ci est défini en tant que serveur (TYPE = serveur). ■ 1 ... 65534 Remarque : lors de la sélection du port, veiller à ce que celui-ci ne soit pas utilisé par d'autres services, comme p. ex. le système d'exploitation. Dans ce cas, aucune connexion ne peut être établie par ce port. Lorsque TYPE = client, le numéro de port est ignoré.

Les réglages pour l'EKI sont définis dans le paragraphe <INTERNAL> ... </INTERNAL> :

Elément	Attribut	Description
ENVIRONMENT	_____	Coupler la suppression de la connexion à des actions (option) ■ Program : suppression après des actions de l'interpréteur robot ■ Remettre le programme à zéro. ■ Abandonner le programme. ■ Reconfigurer les E/S. ■ Submit : suppression après des actions de l'interpréteur Submit ■ Quitter interpréteur Submit. ■ Reconfigurer les E/S. ■ System: suppression après des actions de système ■ Reconfigurer les E/S. Valeur par défaut : Program
BUFFERING	Mode	Procédé avec lequel toutes les mémoires de données sont traitées (option) ■ FIFO: First In First Out ■ LIFO: Last In First Out Valeur par défaut : FIFO
	Limit	Nombre maximum d'éléments de données pouvant être reçus par une mémoire de données (option) ■ 1 ... 512 Valeur par défaut : 16

Élément	Attribut	Description
BUFFSIZE	Limit	Nombre maximum d'octets pouvant être reçus sans être interprétés (option) ■ 1 ... 65534 octets Valeur par défaut : 16384 octets
TIMEOUT	Connect	Durée après laquelle la tentative d'établissement d'une connexion est interrompue (option) Unité : ms ■ 0 ... 65534 ms Valeur par défaut : 2000 ms
ALIVE	Set_Out	Activation d'une sortie ou d'un drapeau lorsque la connexion est établie avec succès (option) Numéro de la sortie : ■ 1 ... 4096 Numéro du drapeau : ■ 1 ... 1024 Tant qu'une connexion avec le système externe est active, la sortie ou le drapeau sont activés. Dès que la connexion avec le système externe est interrompue, la sortie ou le drapeau sont effacés.
	Set_Flag	
	Ping	Intervalle pour l'envoi d'un Ping afin de surveiller la connexion avec le système externe (option) ■ 1 ... 65534 s
IP	———	Adresse IP de l'EKI lorsque celle-ci est définie en tant que serveur (EXTERNAL/TYPE = client). Lorsque EXTERNAL/TYPE = serveur, l'adresse IP est ignorée.
PORT	———	Numéro de port de l'EKI lorsque celle-ci est définie en tant que serveur (EXTERNAL/TYPE = client). ■ 54600 ... 54615 Lorsque EXTERNAL/TYPE = serveur, le numéro de port est ignoré.
PROTOCOL	———	Protocole de transmission (option) ■ TCP ■ UDP Valeur par défaut : TCP Il est recommandé de toujours utiliser le protocole TCP/IP.

Elément	Attribut	Description
MESSAGES	Protocole	Désactiver l'écriture de messages dans la table de messages de l'EKI (option). ■ Warning : les messages d'avertissement et de défauts sont sauvegardés. ■ error: seuls les messages de défauts sont sauvegardés. ■ disabled: le protocole est désactivé. Valeur par défaut : error
	Ecran	Désactiver l'émission de messages sur la smartHMI (option) ■ error: l'émission de messages est active. ■ disabled: l'émission de messages est désactivée. Valeur par défaut : error
	(>>>> 10.3 "Désactivation de l'émission de messages et du protocole de messages" Page 104)	

Exemple

```

<CONFIGURATION>
  <EXTERNAL>
    <IP>172.1.10.5</IP>
    <PORT>60000</PORT>
    <TYPE>Server</TYPE>
  </EXTERNAL>
  <INTERNAL>
    <ENVIRONMENT>Program</ENVIRONMENT>
    <BUFFERING Mode="FIFO" Limit="10"/>
    <BUFFSIZE Limit="16384"/>
    <TIMEOUT Connect="60000"/>
    <ALIVE Set_Out="666" Ping="200"/>
    <IP>192.1.10.20</IP>
    <PORT>54600</PORT>
    <PROTOCOL>TCP</PROTOCOL>
    <MESSAGES Logging="error" Display="disabled"/>
  </INTERNAL>
</CONFIGURATION>

```

6.1.2 Structure XML de la réception de données

Description

La configuration dépend de la réception de données XML ou de données binaires.

- Une structure XML doit être définie pour la réception de données XML : `<XML> ... </XML>`
- Des données brutes doivent être définies pour la réception de données binaires : `<RAW> ... </RAW>`

Attributs dans les éléments de la structure XML `<XML> ... </XML>`:

Élément	Attribut	Description
ELEMENT	Jour	Nom de l'élément La structure XML est définie ici pour la réception des données (XPath). (>>> 6.1.4 "Configuration selon le schéma XPath" Page 31)
ELEMENT	Type	Type de données de l'élément ■ STRING ■ REAL ■ INT ■ BOOL ■ FRAME Remarque : en option, si le tag n'est utilisé que pour les messages d'évènements. Dans ce cas, aucun espace de mémoire n'est réservé pour l'élément. Exemple de drapeau d'évènement : <ELEMENT Tag="Ext" Set_Flag="56"/>
ELEMENT	Set_Out	Activation d'une sortie ou d'un drapeau lorsque l'élément a été reçu (option) Numéro de la sortie : ■ 1 ... 4096 Numéro du drapeau : ■ 1 ... 1024 Remarque : lorsqu'une sortie ou un drapeau est activé(e) avec Set_Out/Set_Flag, la variable de système correspondante \$OUT[]/\$FLAG[] doit être remise à zéro dans le code de programme.
	Set_Flag	
ELEMENT	Mode	Procédé avec lequel un bloc de données est traité dans la mémoire de données ■ FIFO : First In First Out ■ LIFO : Last In First Out N'a d'importance que lorsque des blocs de données individuels doivent être traités différemment, comme cela est configuré pour l'EKI dans BUFFERING.

Attributs pour l'élément dans les données brutes <RAW> ... </RAW> :

Élément	Attribut	Description
ELEMENT	Jour	Nom de l'élément
ELEMENT	Type	Type de données de l'élément ■ BYTE : bloc de données binaires avec longueur fixe ■ STREAM : bloc de données binaires avec séquence de suffixes variable et longueur variable

Elément	Attribut	Description
ELEMENT	Set_Out	Activation d'une sortie ou d'un drapeau lorsque l'élément a été reçu (option) Numéro de la sortie : ■ 1 ... 4096 Numéro du drapeau : ■ 1 ... 1024 Remarque : lorsqu'une sortie ou un drapeau est activé(e) avec Set_Out/Set_Flag, la variable de système correspondante \$OUT[]/\$FLAG[] doit être remise à zéro dans le code de programme.
	Set_Flag	
ELEMENT	EOS	Séquence de suffixes d'une information élémentaire (uniquement important si Type = "STREAM") ■ Codage ASCII : 1 ... 32 caractères ■ Fin alternative séparée par le caractère « ». Exemples : ■ <ELEMENT ... EOS="123,134,21"/> ■ <ELEMENT ... EOS="123,134,21 13,10"/>
ELEMENT	Size	Taille fixe de l'information lorsque Type = "BYTE") ■ 1 ... 3600 octets

Exemples

```

<RECEIVE>
  <XML>
    <ELEMENT Tag="Ext/Str" Type="STRING"/>
    <ELEMENT Tag="Ext/Pos/XPos" Type="REAL" Mode="LIFO"/>
    <ELEMENT Tag="Ext/Pos/YPos" Type="REAL"/>
    <ELEMENT Tag="Ext/Pos/ZPos" Type="REAL"/>
    <ELEMENT Tag="Ext/Temp/Cpu" Type="REAL" Set_Out="1"/>
    <ELEMENT Tag="Ext/Temp/Fan" Type="REAL" Set_Flag="14"/>
    <ELEMENT Tag="Ext/Integer/AState" Type="INT"/>
    <ELEMENT Tag="Ext/Integer/BState" Type="INT"/>
    <ELEMENT Tag="Ext/Boolean/CState" Type="BOOL"/>
    <ELEMENT Tag="Ext/Frames/Frame1" Type="FRAME"/>
    <ELEMENT Tag="Ext/Attributes/@A1" Type="STRING"/>
    <ELEMENT Tag="Ext/Attributes/@A2" Type="INT"/>
    <ELEMENT Tag="Ext" Set_Flag="56"/>
  </XML>
</RECEIVE>

```

```

<RECEIVE>
  <RAW>
    <ELEMENT Tag="RawData" Type="BYTE" Size="1408"
      Set_Flag="14"/>
  </RAW>
</RECEIVE>

```

```

<RECEIVE>
  <RAW>
    <ELEMENT Tag="MyStream" Type="STREAM" EOS="123,134,21"
      Size="836" Set_Flag="14"/>
  </RAW>
</RECEIVE>

```

```
</RAW>
</RECEIVE>
```

6.1.3 Structure XML pour l'envoi de données

Description

La configuration dépend de l'envoi de données XML ou de données binaires.

- Une structure XML doit être définie pour l'envoi de données XML : <XML> ... </XML>



Lors de l'envoi, la structure XML est créée dans même ordre dans lequel elle est configurée.

- L'envoi de données binaires est réalisé directement dans la programmation KRL. Aucune configuration ne doit être indiquée.

Attribut dans les éléments de la structure XML <XML> ... </XML> :

Attribut	Description
Jour	Nom de l'élément La structure XML est définie ici pour l'envoi des données (XPath).

Exemple

```
<SEND>
  <XML>
    <ELEMENT Tag="Robot/Data/ActPos/@X" />
    <ELEMENT Tag="Robot/Data/ActPos/@Y" />
    <ELEMENT Tag="Robot/Data/ActPos/@Z" />
    <ELEMENT Tag="Robot/Data/ActPos/@A" />
    <ELEMENT Tag="Robot/Data/ActPos/@B" />
    <ELEMENT Tag="Robot/Data/ActPos/@C" />
    <ELEMENT Tag="Robot/Status" />
    <ELEMENT Tag="Robot/Mode" />
    <ELEMENT Tag="Robot/Complex/Tickcount" />
    <ELEMENT Tag="Robot/RobotType/Robot/Type" />
  </XML>
</SEND>
```

6.1.4 Configuration selon le schéma XPath

Description

Lorsque XML est utilisé pour l'échange de données, il faut que les documents XML échangés soit structurés selon le même schéma. Pour l'écriture et la lecture des documents XML, KUKA.Ethernet KRL utilise le schéma XPath.

On distingue les cas suivants en fonction du schéma XPath :

- Écriture et lecture d'éléments
- Écriture et lecture d'attributs

Écriture d'éléments

- Document XML sauvegardé pour l'envoi de données :

```
<Robot>
  <Mode>...</Mode>
  <RobotLamp>
    <GrenLamp>
      <LightOn>...</LightOn>
    </GrenLamp>
  </RobotLamp>
</Robot>
```

- Structure XML configurée pour la transmission de données :

```
<SEND>
  <XML>
    <ELEMENT Tag="Robot/Mode" />
    <ELEMENT Tag="Robot/RobotLamp/GrenLamp/LightOn" />
  </XML>
<SEND />
```

Écriture d'attributs

- Document XML sauvegardé pour l'envoi de données :

```
<Robot>
  <Data>
    <ActPos X="...">
  </ActPos>
    <LastPos A="..." B="..." C="..." X="..." Y="..." Z="...">
  </LastPos>
  </Data>
</Robot>
```

- Structure XML configurée pour la transmission de données :

```
<SEND>
  <XML>
    <ELEMENT Tag="Robot/Data/LastPos/@X" />
    <ELEMENT Tag="Robot/Data/LastPos/@Y" />
    ...
    <ELEMENT Tag="Robot/Data/ActPos/@X" />
  </XML>
<SEND />
```

6.2 Fonctions pour l'échange de données

Aperçu

KUKA.Ethernet KRL met des fonctions à disposition pour l'échange de données entre le contrôleur de robot et le système externe.

La description exacte des fonctions figure dans l'annexe. (>>> 10.4 "Référence d'instruction" Page 104)

Initialiser, ouvrir, fermer et effacer une connexion

```
EKI_STATUS = EKI_Init(CHAR[])
EKI_STATUS = EKI_Open(CHAR[])
EKI_STATUS = EKI_Close(CHAR[])
EKI_STATUS = EKI_Clear(CHAR[])
```

Envoyer des données

```
EKI_STATUS = EKI_Send(CHAR[], CHAR[], INT)
```

Écrire des données

```
EKI_STATUS = EKI_SetReal(CHAR[], CHAR[], REAL)
EKI_STATUS = EKI_SetInt(CHAR[], CHAR[], INT)
EKI_STATUS = EKI_SetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_SetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS = EKI_SetString(CHAR[], CHAR[], CHAR[])
```

Lire des données

```
EKI_STATUS = EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
```

Lire des données	
EKI_STATUS =	EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS =	EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS =	EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS =	EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS =	EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS =	EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS =	EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])
Vérifier que la fonction est exempte de défauts	
EKI_CHECK(EKI_STATUS, EKriMsgType, CHAR[])	
Effacer, verrouiller, déverrouiller et contrôler la mémoire de données	
EKI_STATUS =	EKI_Clear(CHAR[])
EKI_STATUS =	EKI_ClearBuffer(CHAR[], CHAR[])
EKI_STATUS =	EKI_Lock(CHAR[])
EKI_STATUS =	EKI_Unlock(CHAR[])
EKI_STATUS =	EKI_CheckBuffer(CHAR[], CHAR[])

6.2.1 Astuces de programmation

- Lorsqu'une connexion est établie dans l'interpréteur Submit, il faut observer les points suivants :
 - Dans la configuration de connexion, il faut indiquer qu'il s'agit d'un canal Submit avec l'élément <ENVIRONMENT>.
 - Un canal ouvert dans l'interpréteur Submit peut également être activé par l'interpréteur robot.
 - Lorsque l'interpréteur Submit est abandonné, la connexion par configuration est effacée automatiquement.
- Lorsque l'interpréteur Submit et l'interpréteur robot ont accès parallèlement à un canal, il faut observer les points suivants :
 - Lorsque les données sont lues avec EKI_Get...(), le contenu de cette mémoire est effacée. C'est la raison pour laquelle il n'est pas sensé de lire les données parallèlement avec l'interpréteur Submit et l'interpréteur robot.
 - L'interpréteur Submit et l'interpréteur robot peuvent avoir accès en écriture au même canal avec EKI_Set...(). Ce faisant, les appels peuvent écraser mutuellement les données lorsqu'il y a accès au même élément cible et que EKI_Send() n'est pas appelé entre deux actions.
 - Les moments d'exécution dans les différents interpréteurs peuvent changer les uns par rapport aux autres, ce qui peut entraîner une modification inattendue du comportement lors d'accès en écriture ou en lecture concurrents.



Il est recommandé d'ouvrir un canal individuel pour chaque tâche d'envoi et de réception.



Le comportement d'envoi et de réception décrit est également valable lorsque KUKA.MultiSubmitInterpreter est utilisé.



Les instructions EKI sont exécutées à l'avance !

- Si on souhaite exécuter une instruction EKI pendant la séquence principale, il faut utiliser des instructions déclenchant un stop à l'avance, p. ex. WAIT SEC.
- Chaque accès à la mémoire prenant du temps, il est recommandé d'appeler de grandes quantités de données avec les fonctions d'accès aux champs EKI_Get...Array().
- KUKA.Ethernet KRL peut avoir accès à 512 éléments de champs maximum avec EKI_Get...Array(). Des champs plus grands peuvent être créés en KRL, p. ex. myFrame[1000], mais seuls 512 éléments maximum peuvent être lus.
- Il y a différentes possibilités d'attendre des données :
 - En activant un drapeau ou une sortie, il est possible d'afficher qu'un élément défini ou un bloc de données complet a été reçu. (Élément Set_Flag ou Set_Out dans la structure XML pour la réception de données).

Exemple : interruption du programme KRL avec WAIT FOR \$FLAG[x]
(>>> 7.7 "Exemple « XmlCallback »" Page 55)

- La fonction EKI_CheckBuffer() permet d'interroger de façon cyclique si de nouveaux éléments de données se trouvent dans la mémoire.

6.2.2 Initialiser et supprimer une connexion

Description

Une connexion doit être créée et initialisée avec la fonction EKI_Init(). La configuration de connexion indiquée dans la fonction est lue ce faisant. Une connexion peut être créée dans l'interpréteur robot comme dans l'interpréteur Submit.

Une connexion peut être à nouveau effacée dans l'interpréteur robot ou Submit avec la fonction EKI_Clear(). De plus, la suppression d'une connexion peut être reliée à des actions d'interpréteur robot et Submit ou des actions de système. (peut être configuré avec l'élément <ENVIRONMENT> dans la configuration de connexion)

Configuration « Program »

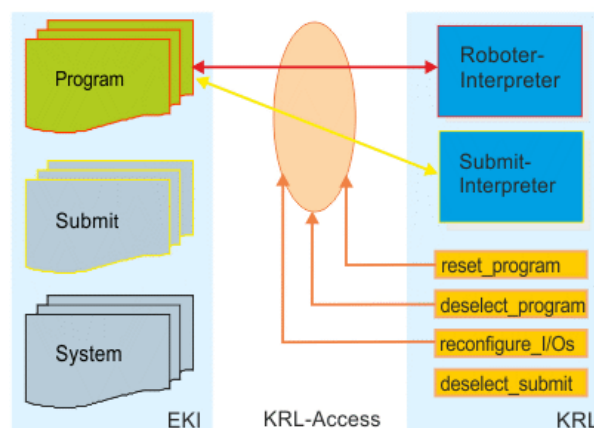


Fig. 6-1: Configuration de connexion « Program »

Avec cette configuration, une connexion est supprimée après les actions suivantes.

- Remettre le programme à zéro.
- Abandonner le programme.
- Reconfigurer les E/S.

i Lors de la reconfiguration des E/S, le driver est chargé à nouveau et toutes les initialisations sont effacées.

Configuration « Submit »

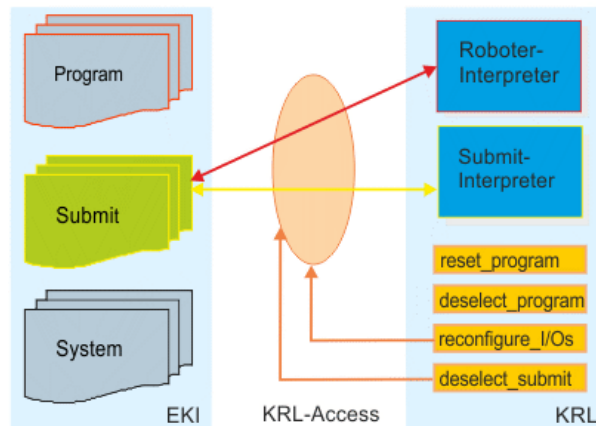


Fig. 6-2: Configuration de connexion « Submit »

Avec cette configuration, une connexion est supprimée après les actions suivantes.

- Quitter interpréteur Submit.
- Reconfigurer les E/S.

i Lorsque KUKA.MultiSubmitInterpreter est utilisé, la connexion ne peut être coupée qu'une fois l'interpréteur System-Submit abandonné. Les interpréteurs Submit disponibles en plus de l'interpréteur System-Submit n'ont aucune influence sur la coupure de la connexion.

i Lors de la reconfiguration des E/S, le driver est chargé à nouveau et toutes les initialisations sont effacées.

Configuration « System »

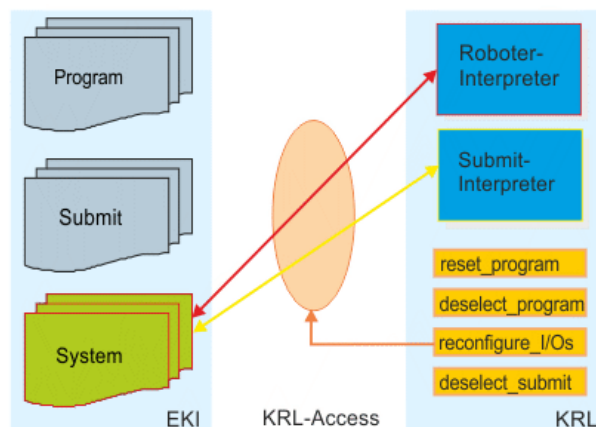


Fig. 6-3: Configuration de connexion « System »

Avec cette configuration, une connexion est supprimée après les actions suivantes.

- Reconfigurer les E/S.

i Lors de la reconfiguration des E/S, le driver est chargé à nouveau et toutes les initialisations sont effacées.

6.2.3 Ouvrir et fermer une connexion

Description

La connexion avec un système externe est établie avec un programme KRL. A plupart des programmes KRL sont structurés de la façon suivante :

```

1 DEF Connection()
2 ...
3 RET=EKI_Init("Connection")
4 RET=EKI_Open("Connection")
5 ...
6 Write data, send data or get received data
7 ...
8 RET=EKI_Close("Connection")
9 RET=EKI_Clear("Connection")
10 ...
11 END

```

Ligne	Description
3	EKI_Init() initialise le canal avec lequel l'EKI se connecte avec le système externe.
4	EKI_Open() ouvre le canal.
6	Instructions KRL pour écrire des données dans la mémoire, envoyer des données ou avoir accès à des données reçues.
8	EKI_Close() ferme le canal.
9	EKI_Clear() efface le canal.

Lors de la programmation, il faut prendre en compte si l'EKI est configuré en tant que serveur ou en tant que client.

Mode serveur

Lorsque le système externe est configuré en tant que client, EKI_Open() fait passer EKI (= serveur) en état d'interception. Le serveur attend la demande de connexion d'un client sans que le déroulement du programme soit interrompu. Si l'élément <TIMEOUT Connect="..."/> n'est pas décrit dans le fichier de configuration, le serveur attend jusqu'à ce que le client demande une connexion.

Une demande de connexion par un client est signalée par un accès à l'EKI ou par un message d'évènement, p. ex. avec l'élément <ALIVE SET_OUT="..."/>.

Si on souhaite interrompre le déroulement du programme tant que le serveur attend la demande de connexion, un drapeau ou une sortie d'évènement doit être programmé(e), p. ex. WAIT FOR \$OUT[...].



Il est recommandé de ne pas utiliser EKI_Close() en mode serveur. En mode serveur, le canal est fermé par le client externe.

Mode client

Lorsque le système externe est configuré en tant que serveur, EKI_Open() interrompt le déroulement du programme jusqu'à ce que la connexion avec le système externe soit active. EKI_Close() coupe la connexion avec le serveur externe.

6.2.4 Envoi de données

Description

En fonction de la configuration et de la programmation, il est possible d'envoyer les données suivantes avec EKI_Send() :

Pour une communication via structure XML :

- Structure XML complète

- Partie d'une structure XML
- Chaîne de caractères quelconque de longueur variable

Pour une communication via données brutes :

- Pour des blocs de données binaires avec longueur fixe (attribut `Type= « BYTE »`) : chaîne de caractères quelconque de longueur fixe
 - La longueur de la chaîne de caractères en octets doit correspondre exactement à l'attribut configuré `Size`. En cas de dépassement, un message de défaut est émis, si la longueur n'est pas atteinte, un message d'avertissement est émis.
 - Les blocs de données binaires de longueur fixe doivent être lus dans le programme KRL avec `CAST_TO()`. Seules les données de type `REAL` (4 octets) peuvent être lues, non celles de type `Double`.



Voir la documentation CREAD/CWRITE pour obtenir des informations plus détaillées sur l'instruction `CAST_TO()`.

- Pour des blocs de données binaires avec séquence de suffixes variable (attribut `Type= « STREAM »`) : chaîne de caractères quelconque de longueur variable
La séquence de suffixes est envoyée automatiquement.

Exemples

Envoi de données XML :

Envoi d'une structure XML complète :

- Structure XML sauvegardée pour l'envoi de données :

```
<Robot>
  <ActPos X="1000.12"></ActPos>
  <Status>12345678</Status>
</Robot>
```

- Programmation :

```
DECL EKI_STATUS RET
RET=EKI_Send("Channel_1", "Robot")
```

- Structure XML envoyée :

```
<Robot>
  <ActPos X="1000.12"></ActPos>
  <Status>12345678</Status>
</Robot>
```

Envoi d'une partie de la structure XML :

- Structure XML sauvegardée pour l'envoi de données :

```
<Robot>
  <ActPos X="1000.12"></ActPos>
  <Status>12345678</Status>
</Robot>
```

- Programmation :

```
DECL EKI_STATUS RET
RET=EKI_Send("Channel_1", "Robot/ActPos")
```

- Structure XML envoyée :

```
<Robot>
  <ActPos X="1000.12"></ActPos>
</Robot>
```

Envoi direct d'une chaîne de caractères :

- Structure XML sauvegardée pour l'envoi de données (n'est pas utilisée pour l'envoi direct) :

```
<Robot>
  <ActPos X="1000.12"></ActPos>
  <Status>12345678</Status>
</Robot>
```

- Programmation :

```
DECL EKI_STATUS RET
RET=EKI_Send("Channel_1", "<POS><XPOS>1</XPOS></POS>")
```

- Chaîne de caractères envoyée :

```
<POS><XPOS>1</XPOS></POS>
```

Envoi de données binaires :

Envoi d'un bloc de données binaires avec longueur fixe (10 octets) :

- Données brutes configurées :

```
<RAW>
  <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
</RAW>
```

- Programmation :

```
DECL EKI_STATUS RET
CHAR Bytes[10]
OFFSET=0
CAST_TO(Bytes[], OFFSET, 91984754, 913434.2, TRUE, "X")
RET=EKI_Send("Channel_1", Bytes[])
```

- Données envoyées :

```
"r?{ ? _I X"
```

Envoi d'un bloc de données binaires avec séquence de suffixes et longueur variable :

- Données brutes configurées :

```
<RAW>
  <ELEMENT Tag="Buffer" Type="STREAM" EOS="65,66" />
</RAW>
```

- Programmation :

```
DECL EKI_STATUS RET
CHAR Bytes[64]
Bytes[]="Stream ends with:"
RET=EKI_Send("Channel_1", Bytes[])
```

- Données envoyées :

```
"Stream ends with:AB"
```

Envoi d'un bloc de données binaires avec limitation de caractères et avec séquence de suffixes et longueur variable :

- Données brutes configurées :

```
<RAW>
  <ELEMENT Tag="Buffer" Type="STREAM" EOS="65,66" />
</RAW>
```

- Programmation :

```
DECL EKI_STATUS RET
CHAR Bytes[64]
```

```
Bytes[]="Stream ends with:"
RET=EKI_Send("Channel_1",Bytes[],6)
```

- Données envoyées :

```
"StreamAB"
```

6.2.5 Lecture de données



Pour la lecture de données, il faut initialiser les variables KRL correspondantes, p. ex. en affectant des valeurs.

Description

Lors de la sauvegarde et la lecture des données, les données XML et binaires sont traitées différemment :

- Les données XML sont extraites par l'EKI et sauvegardées dans différentes mémoires en fonction de leur type. Il est possible d'avoir accès individuellement à chaque valeur sauvegardée.
Pour lire les données, XML, il est possible d'utiliser toutes les fonctions d'accès EKI_Get...().
- Les blocs de données binaires ne sont pas interprétés par l'EKI et sauvegardés en tant qu'ensemble dans une mémoire.
Pour lire un bloc de données binaires dans une mémoire, il faut utiliser la fonction d'accès EKI_GetString(). Les blocs de données binaires sont lus dans la mémoire en tant que séquences de caractères.
Les blocs de données binaires de longueur fixe doivent être à nouveau divisés en variables individuelles dans le programme KRL avec CAST_FROM().



Voir la documentation CREAD/CWRITE pour obtenir des informations plus détaillées sur l'instruction CAST_FROM().

Exemples

Lecture de données XML :

Structure XML pour la réception de données

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmation :

```
; Declaration
INT i
DECL EKI_STATUS RET
CHAR valueChar[256]
BOOL valueBOOL

; Initialization
FOR i=(1) TO (256)
  valueChar[i]=0
ENDFOR
valueBOOL=FALSE

RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Document XML écrit avec des données de capteur (données reçues) :

```
<Sensor>
  <Message>Example message</Message>
  <Status>
    <IsActive>1</IsActive>
  </Status>
</Sensor>
```

Lecture d'un bloc de données binaires avec longueur fixe (10 octets) :

■ Données brutes configurées :

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

■ Programmation :

```
; Declaration
INT i
INT OFFSET
DECL EKI_STATUS RET
CHAR Bytes[10]
INT valueInt
REAL valueReal
BOOL valueBool
CHAR valueChar[1]

; Initialization
FOR i=(1) TO (10)
  Bytes[i]=0
ENDFOR

OFFSET=0
valueInt=0
valueBool=FALSE
valueReal=0
valueChar[1]=0

RET=EKI_GetString("Channel_1", "Buffer", Bytes[])

OFFSET=0
CAST_FROM(Bytes[],OFFSET,valueReal,valueInt,valueChar[],valueBool)
```

Lecture d'un bloc de données binaires avec séquence de suffixes :

■ Données brutes configurées :

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="STREAM" EOS="13,10" />
  </RAW>
</RECEIVE>
```

■ Programmation :

```
; Declaration
INT i
DECL EKI_STATUS RET
CHAR Bytes[64]

; Initialization
FOR i=(1) TO (64)
```

```

    Bytes[i]=0
ENDFOR

RET=EKI_GetString("Channel_1", "Buffer", Bytes[])

```

6.2.6 Suppression de mémoires de données

Description

Lors de la suppression de mémoires de données, il faut différencier les cas suivants :

- Effacer avec EKI_Clear() : effacer toutes les mémoires de données et fermer la connexion Ethernet
- Effacer avec EKI_ClearBuffer() : effacer certaines mémoires de données sans fermer la connexion Ethernet

EKI_ClearBuffer()

Les mémoires de données pouvant être effacées avec l'instruction EKI_ClearBuffer() dépendent de la communication via des données brutes ou via une structure XML.

Pour une communication via données brutes :

- La mémoire de données pour données reçues peut être effacée.

Pour une communication via structure XML :

- La mémoire de données d'un élément peut être effacée avec l'indication de l'impression XPath d'un élément de la structure XML configurée. Cela concerne aussi bien les données reçues que les données à envoyer.



Les données XML sont extraites par l'EKI et sauvegardées dans différentes mémoires en fonction de leur type. Lorsque des mémoires individuelles sont effacées, il faudra s'assurer que des données en rapport les unes avec les autres ne sont pas perdues.

Exemples

Communication via données brutes :

Suppression de la mémoire de données pour données reçues :

- Données brutes configurées :

```

<RECEIVE>
  <RAW>
    <ELEMENT Tag="RawData" Type="BYTE" Size="1408" Set_Flag="14"/>
  </RAW>
</RECEIVE>

```

- Programmation :

```

DECL EKI_STATUS RET
RET=EKI_ClearBuffer("Channel_1", "RawData")

```

Communication via structure XML :

Suppression de la mémoire de données du tag <Flag> :

- Structure XML configurée pour la réception de données :

```

<RECEIVE>
  <XML>
    <ELEMENT Tag="Ext/Activ/Value" Type="REAL"/>
    <ELEMENT Tag="Ext/Activ/Flag" Type="BOOL"/>
    <ELEMENT Tag="Ext/Activ/Flag/Message" Type="STRING"/>
  </XML>
</RECEIVE>

```

La mémoire de données du tag <Message> est également effacée car ce tag est subordonnée au tag <Flag>.

■ Programmation :

```
DECL EKI_STATUS RET
RET=EKI_ClearBuffer("Channel_1", "Ext/Activ/Flag")
```

6.2.7 EKI_STATUS – Structure pour valeurs renvoyées spécifiques à la fonction

Description Chaque fonction de l'EKI renvoie des valeurs spécifiques à la fonction. EKI_STATUS est la variable de structure globale dans laquelle ces valeurs sont écrites.

Syntaxe GLOBAL STRUC EKI_STATUS INT Buff, Read, Msg_No, BOOL Connected, INT Counter

Explication de la syntaxe

Élément	Description
Buff	Nombre d'éléments se trouvant encore dans la mémoire après un accès.
Read	Nombre d'éléments lus dans la mémoire
Msg_No	Numéro du défaut apparu lors de l'appel d'une fonction ou lors de la réception de données. Lorsque l'émission automatique de messages est désactivée, EKI_CHECK() peut être utilisée pour lire le numéro de défaut et émettre le message de défaut sur la smartHMI.
Connected	Indique s'il y a une connexion ■ TRUE = connexion existante ■ FALSE = connexion interrompue
Counter	Horodatage pour paquets de données reçues Les paquets de données arrivant dans la mémoire sont numérotés en continu dans l'ordre dans lequel ils sont sauvegardés dans la mémoire. Lorsque des données individuelles sont lues, l'élément de structure Counter se voit affecter l'horodatage du paquet de données duquel l'élément de données provient. (>>> 6.2.10 "Édition de blocs de données incomplets" Page 44)

Valeurs renvoyées Les éléments suivants de la structure EKI_STATUS sont écrits selon la fonction :

Fonction	Buff	Read	Msg_No	Connected	Counter
EKI_Init()	✗	✗	✓	✗	✗
EKI_Open()	✗	✗	✓	✓	✗
EKI_Close()	✗	✗	✓	✗	✗
EKI_Clear()	✗	✗	✓	✗	✗
EKI_Send()	✗	✗	✓	✓	✗
EKI_Set...()	✗	✗	✓	✓	✗
EKI_Get...()	✓	✓	✓	✓	✓

Fonction	Buff	Read	Msg_No	Connec- ted	Counter
EKI_ClearBuffer()	✗	✗	✓	✓	✗
EKI_Lock()	✗	✗	✓	✓	✗
EKI_UnLock()	✗	✗	✓	✓	✗
EKI_CheckBuffer()	✓	✗	✓	✓	✓

6.2.8 Configuration de messages d'évènements

L'activation d'une sortie ou d'un drapeau permet de signaler les évènements suivants :

- La connexion est active.
- Un élément XML individuel est arrivé.
- Une structure XML complète ou un bloc de données binaires complet sont arrivés.

Sortie d'évènement

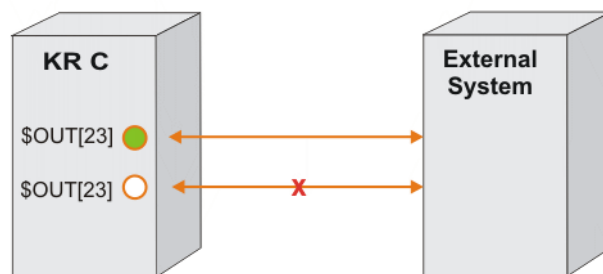


Fig. 6-4: Sortie d'évènement (connexion active)

\$OUT[23] est activé tant que la connexion avec le système externe est active. Si la connexion n'est plus active, \$OUT[23] est remis à zéro.

i La connexion ne peut être rétablie qu'avec la fonction EKI_OPEN().

Drapeau d'évènement

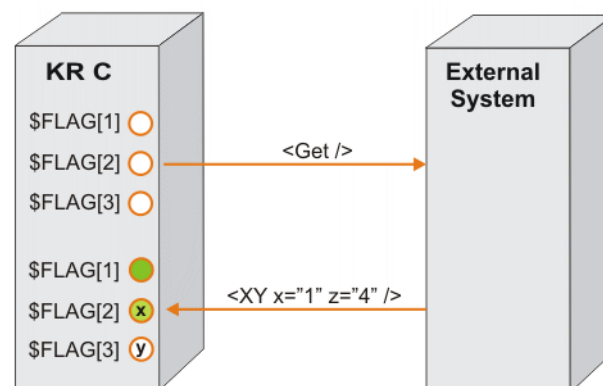


Fig. 6-5: Drapeau d'évènement (structure XML complète)

La structure XML `<XY />` contient les éléments de données `"XY/@x"` et `"XY/@z"`. \$FLAG[1] est activé car la structure XML complète est arrivée. \$FLAG[2] est activé car « XY » contient l'élément « x ». \$FLAG[3] n'est pas activé car l'élément « y » n'a pas été transmis.

Exemple

(>>> 7.7 "Exemple « XmlCallback »" Page 55)

6.2.9 Réception de blocs de données XML complets

Description

Les fonctions d'accès EKI_Get...() sont verrouillées jusqu'à ce que toutes les données d'un bloc de données XML se trouvent dans la mémoire.

Lorsque LIFO est configuré et que 2 blocs de données XML ou plus arrivent directement l'un après l'autre, il n'est plus garanti qu'un bloc de données soit cherché de la mémoire en rapport avec un autre. Il peut p. ex. arriver que les données du deuxième bloc de données soient déjà sauvegardées dans la mémoire alors que le premier bloc de données n'est pas encore entièrement traité. En mode LIFO, on a toujours accès d'abord aux dernières données sauvegardées, c'est pourquoi le bloc de données disponibles en KRL est incohérent.

Afin d'éviter la fragmentation de blocs de données en mode LIFO, il faut verrouiller le traitement de données nouvellement reçues jusqu'à ce que toutes les données en rapport les unes avec les autres aient été cherchées dans la mémoire.

Exemple

```
...
RET=EKI_Lock ("MyChannel")
RET=EKI_Get... ()
RET=EKI_Get... ()
...
RET=EKI_Get... ()
RET=EKI_Unlock ("MyChannel")
...
```

6.2.10 Édition de blocs de données incomplets

Il peut arriver qu'un système externe envoie des blocs de données incomplets. Des éléments XML individuels sont vides ou manquent entièrement ; des données de différents paquets de données se trouvent donc dans une surface de mémoire.

Si les blocs de données doivent être en rapport les uns avec les autres en KRL, il est possible d'utiliser l'élément de structure Counter des variables EKI_STATUS. Lorsque les fonctions de EKI_Get...Array sont utilisées, des données n'ayant pas de rapport temporel les unes avec les autres sont reconnues lorsque qu'il y a renvoi de Counter = 0.

6.2.11 EKI_CHECK() – Vérifier que les fonctions sont exemptes de défauts

Description

KUKA.Ethernet KRL émet un message sur la smartHMI pour chaque défaut. L'émission automatique de messages peut être désactivée.

(>>> 10.3 "Désactivation de l'émission de messages et du protocole de messages" Page 104)

Lorsque l'émission automatique de messages est désactivée, il est recommandé d'utiliser EKI_CHECK() pour vérifier si un défaut s'est produit lors de l'exécution d'une fonction :

- Le numéro de défaut est lu et le message correspondant est émis sur la smartHMI.
- Si un nom de canal est indiqué dans EKI_CHECK(), il y aura interrogation de la présence de défauts lors de la réception de données.

(>>> 10.4.5 "Vérifier que la fonction est exempte de défauts" Page 112)

Lors de chaque appel d'EKI_CHECK(), le programme KRC:\R1\TP\EthernetKRL\EthernetKRL_USER.SRC est appelé. Dans ce programme, il est possible de programmer des réactions aux défauts spécifiques à l'utilisateur.

Exemple

Une connexion est toujours coupée lorsqu'un défaut apparaît à la réception. Une interruption peut être programmée en tant que stratégie de défaut en cas d'interruption de la connexion Ethernet.

- Dans le fichier de configuration XmlTransmit.XML, on définit que FLAG[1] est activé en cas de connexion établie avec succès. En cas de perte de connexion, FLAG[1] est remis à zéro.

```
<ALIVE Set_Flag="1"/>
```

- L'interruption est déclarée et activée dans le programme KRL. Si FLAG[1] est remis à zéro, le programme d'interruption est exécuté.

```
;FOLD Define callback
  INTERRUPT DECL 89 WHEN $FLAG[1]==FALSE DO CON_ERR()
  INTERRUPT ON 89
;ENDFOLD
```

- Dans le programme d'interruption, EKI_CHECK() permet d'interroger le défaut apparu et de rétablir la connexion.

```
DEF CON_ERR()
  DECL EKI_STATUS RET
  RET={Buff 0,Read 0, Msg_no 0, Connected false}
  EKI_CHECK(RET,#Quit,"XmlTransmit")
  RET=EKI_OPEN("XmlTransmit")
END
```


7 Configurations et de programmes à titre d'exemple

Un programme de serveur ainsi que différents exemples de configuration et de programme font partie de l'étendue livraison de KUKA.Ethernet KRL. Ces exemples de configuration et de programme permettent d'établir une communication entre le programme de serveur et le contrôleur de robot.

Composants	Répertoire
Programme de serveur ■ EthernetKRL_Server.exe	DOC\Example\Application
Exemples de programme en KRL ■ BinaryFixed.src ■ BinaryStream.src ■ XmlCallback.src ■ XmlServer.src ■ XmlTransmit.src	DOC\Example\Program
Exemples de configuration en XML ■ BinaryFixed.xml ■ BinaryStream.xml ■ XmlCallBack.xml ■ XmlServer.xml ■ XmlTransmit.xml ■ XmlFullConfig.xml	DOC\Example\Config

7.1 Intégration d'un programme de serveur et d'exemples

Condition préalable

Système externe :

- Système d'exploitation Windows avec .NET-Framework 3.5 ou une version ultérieure

Contrôleur de robot :

- Groupe d'utilisateurs Expert
- Mode T1 ou T2.

Procédure

1. Copier le programme de serveur sur un système externe.
2. Insérer l'exemple de configuration XML dans le répertoire C:\KRC\Roboter\Config\User\Common\EthernetKRL du contrôleur de robot.
3. Insérer les exemples de programmes KRL dans le répertoire C:\KRC\ROBOTER\Program du contrôleur de robot.
4. Démarrer le programme de serveur sur le système externe.
(>>> 7.2 "Interface utilisateur du programme de serveur" Page 48)
5. Actionner le bouton de menu. La fenêtre **Communication Properties** s'ouvre.
(>>> 7.2.1 "Réglage des paramètres de communication dans le programme de serveur" Page 49)
6. Uniquement si plusieurs interfaces réseau sont disponibles au système externe : saisir le numéro de l'adaptateur réseau (= index de carte réseau) utilisé pour la communication avec le contrôleur de robot.
7. Fermer la fenêtre **Communication Properties** et appuyer sur le bouton Start. L'adresse IP disponible pour la communication est affichée dans la fenêtre de messages.

8. Régler l'adresse IP affichée du système externe dans le fichier XML souhaité.

7.2 Interface utilisateur du programme de serveur

Description

Le programme de serveur permet de tester la communication entre un système externe et le contrôleur de robot en établissant une communication stable vers le contrôleur de robot.

Le programme de serveur contient les fonctionnalités suivantes :

- Envoi et réception de données (automatiquement ou manuellement)
- Affichage des données reçues
- Affichage des données envoyées

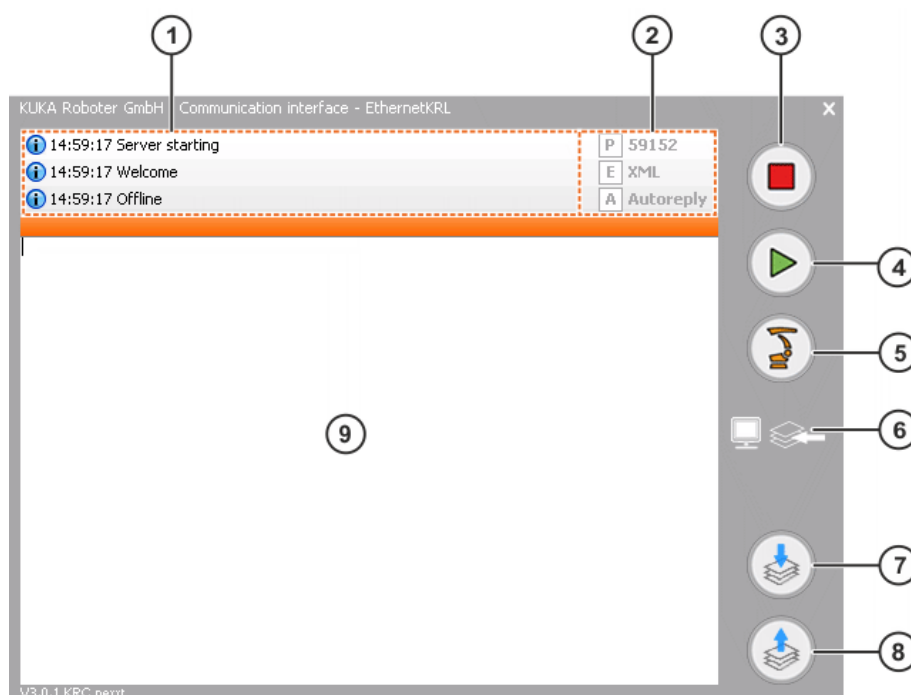


Fig. 7-1: Interface utilisateur du programme de serveur

Pos.	Description
1	Fenêtre de messages
2	Affichage des paramètres de communication réglés (>>> 7.2.1 "Réglage des paramètres de communication dans le programme de serveur" Page 49) ■ P : Numéro de port ■ E: Données d'exemple ■ Xml: données XML ■ BinaryFixed: données binaires avec longueur fixe ■ BinaryStream: flux de données binaires variables avec séquence de suffixes ■ A: mode Communication ■ Autoreply: le serveur répond automatiquement à chaque paquet de données reçu. ■ Manual: Uniquement réception manuelle de données ou envoi manuel de données

Pos.	Description
3	Bouton Stop La communication avec la commande du robot est terminée et le serveur est remis à zéro.
4	Bouton Start L'échange entre le programme de serveur et la commande du robot est analysé. La première demande de connexion est accordée et utilisée en tant qu'adaptateur de communication.
5	Bouton de menu permettant de régler les paramètres de communication (>>> 7.2.1 "Réglage des paramètres de communication dans le programme de serveur" Page 49)
6	Options d'affichage ■ Flèche pointée vers la gauche : les données reçues sont affichées. (par défaut). ■ Flèche pointée vers la droite : les données transmises sont affichées.
7	Bouton pour la réception manuelle de données
8	Bouton pour l'envoi manuel de données
9	Fenêtre d'affichage Les données transmises ou reçues sont affichées en fonction de l'option d'affichage choisie.

7.2.1 Réglage des paramètres de communication dans le programme de serveur

Procédure

1. Cliquer sur le bouton Menu dans le programme de serveur.
La fenêtre **Communication Properties** s'ouvre.
2. Régler les paramètres de communication.
3. Fermer la fenêtre.

Description

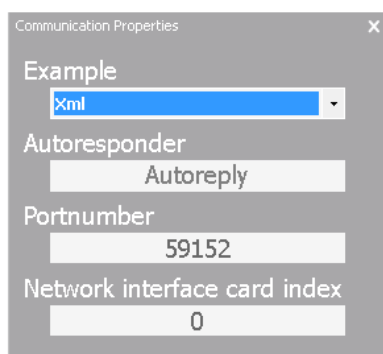


Fig. 7-2: Fenêtre Communication Properties

Paramètre	Description
Exemple	Sélectionner les données d'exemple. ■ Xml: données XML ■ BinaryFixed: données binaires avec longueur fixe ■ BinaryStream: flux de données binaires variables avec séquence de suffixes Valeur par défaut : xml
Autoresponder	Sélectionner le mode Communication. ■ Autoreply: le serveur répond automatiquement à chaque paquet de données reçu. ■ Manual: Uniquement réception manuelle de données ou envoi manuel de données Valeur par défaut : Autoreply
Numéro de port	Entrer le numéro de port de la connexion de socket. Le système externe attend la demande de connexion de la commande du robot sur ce port. Sélectionner un numéro libre n'étant pas occupé en service standard. Valeur par défaut : 59152 Remarque : lors de la sélection du port, veiller à ce que celui-ci ne soit pas utilisé par d'autres services, comme p. ex. le système d'exploitation. Dans ce cas, aucune connexion ne peut être établie par ce port.
Network interface card index	Saisir le numéro de l'adaptateur réseau. N'a d'importance que si le système externe utilise plusieurs cartes réseau, p. ex. WLAN et LAN. Valeur par défaut : 0

7.3 Exemple « BinaryFixed »



Pour la communication avec le contrôleur de robot, il est nécessaire de régler les données d'exemple correspondantes dans le programme du serveur, ici, « BinaryFixed ».

EKI est configurée en tant que client. Seuls des blocs de données binaires avec une longueur fixe de 10 octets et portant le nom d'élément « Buffer » peuvent être reçus avec cette connexion. Le programme de serveur envoie un bloc de données. Lorsqu'EKI a reçu des données externes, \$FLAG[1] est activé.

Fichier XML

```
<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>59152</PORT>
    </EXTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <RAW>
      <ELEMENT Tag="Buffer" Type="BYTE" Set_Flag="1" Size="10" />
    </RAW>
  </RECEIVE>
  <SEND/>
</ETHERNETKRL>
```

Les blocs de données binaires de longueur fixe doivent être saisis et lus dans le programme KRL avec CAST_TO() et CAST_FROM(). Seules les données de type REAL (4 octets) peuvent être lues, non celles de type Double.



Voir la documentation CREAD/CWRITE pour obtenir des informations plus détaillées sur les instructions CAST_TO() et CAST_FROM().

Programme

```

1 DEF BinaryFixed( )
2 Declaration
3 INI
4 Initialize sample data
5
6 RET=EKI_Init("BinaryFixed")
7 RET=EKI_Open("BinaryFixed")
8
9 OFFSET=0
10 CAST_TO(Bytes[],OFFSET,34.425,674345,"R",TRUE)
11
12 RET=EKI_Send("BinaryFixed",Bytes[])
13
14 WAIT FOR $FLAG[1]
15 RET=EKI_GetString("BinaryFixed","Buffer",Bytes[])
16 $FLAG[1]=FALSE
17
18 OFFSET=0
19 CAST_FROM(Bytes[], OFFSET, valueReal, valueInt, valueChar[],
    valueBool)
20
21 RET=EKI_Close("BinaryFixed")
22 RET=EKI_Clear("BinaryFixed")
23 END

```

Ligne	Description
4	Initialisation des variables KRL par affectation de valeurs
6	EKI_Init() initialise le canal avec lequel l'EKI se connecte avec le système externe.
7	EKI_Open() ouvre le canal et se connecte avec le serveur.
9, 10	CAST_TO écrit les valeurs dans le champ CHAR octets[].
12	EKI_Send() envoie le champ CHAR octets[] au système externe.
14 ... 16	\$FLAG[1] signale la réception de l'élément de données configuré. EKI_GetString accède à la mémoire et copie les données dans le champ CHAR octets[]. \$FLAG[1] est remis à zéro.
18, 19	CAST_FROM lit les données comprises dans le champ CHAR octets[] et les copie dans les variables indiquées en fonction de leur type.
21	EKI_Close() ferme le canal.
22	EKI_Clear() efface le canal.

7.4 Exemple « BinaryStream »



Pour la communication avec le contrôleur de robot, il est nécessaire de régler les données d'exemple correspondantes dans le programme du serveur, ici, « BinaryStream ».

EKI est configurée en tant que client. Seuls des blocs de données binaires avec une longueur maximum de 64 octets et portant le nom d'élément « Buffer » peuvent être reçus avec cette connexion. La fin du bloc de données binaires doit être identifié avec la séquence de suffixes CR, LF. Lorsqu'EKI a reçu cet élément, \$FLAG[1] est activé.

Fichier XML

```
<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>59152</PORT>
    </EXTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <RAW>
      <ELEMENT Tag="Buffer" Type="STREAM" Set_Flag="1"
        Size="64" EOS="13,10" />
    </RAW>
  </RECEIVE>
  <SEND/>
</ETHERNETKRL>
```

Programme

```
1 DEF BinaryStream( )
2 Declaration
3 INI
4 Initialize sample data
5
6 RET=EKI_Init("BinaryStream")
7 RET=EKI_Open("BinaryStream")
8
9 Bytes[]="Stream ends with CR,LF"
10
11 RET=EKI_Send("BinaryStream",Bytes[])
12
13 WAIT FOR $FLAG[1]
14 RET=EKI_GetString("BinaryStream","Buffer",Bytes[])
15 $FLAG[1]=FALSE
16
17 RET=EKI_Close("BinaryStream")
18 RET=EKI_Clear("BinaryStream")
19
20 END
```

Ligne	Description
4	Initialisation des variables KRL par affectation de valeurs
6	EKI_Init() initialise le canal avec lequel l'EKI se connecte avec le système externe.
7	EKI_Open() ouvre le canal et se connecte avec le serveur.
9	Des données sont écrites dans le champ CHAR octets[].
11	EKI_Send() envoie le champ CHAR octets[] au système externe.

Ligne	Description
13 ... 15	\$FLAG[1] signale la réception de l'élément de données configuré. EKI_GetString lit la chaîne de caractères dans le champ CHAR octets[] dans la mémoire. \$FLAG[1] est remis à zéro.
17	EKI_Close() ferme le canal.
18	EKI_Clear() efface le canal.

7.5 Exemple « XmlTransmit »



Pour la communication avec le contrôleur de robot, il est nécessaire de régler les données d'exemple correspondantes dans le programme du serveur, ici, « Xml ».

EKI est configurée en tant que client. Les données de robot sont envoyées et les données de capteur sont lues dans la mémoire après écoulement d'un temps d'attente d'1 sec.

Fichier XML

```
<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>59152</PORT>
    </EXTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <XML>
      <ELEMENT Tag="Sensor/Message" Type="STRING" />
      <ELEMENT Tag="Sensor/Positions/Current/@X" Type="REAL" />
      <ELEMENT Tag="Sensor/Positions/Before/X" Type="REAL" />
      <ELEMENT Tag="Sensor/Nmb" Type="INT" />
      <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
      <ELEMENT Tag="Sensor/Read/xyzabc" Type="FRAME" />
      <ELEMENT Tag="Sensor/Show/@error" Type="BOOL" />
      <ELEMENT Tag="Sensor/Show/@temp" Type="INT" />
      <ELEMENT Tag="Sensor/Show" Type="STRING" />
      <ELEMENT Tag="Sensor/Free" Type="INT" />
    </XML>
  </RECEIVE>
  <SEND>
    <XML>
      <ELEMENT Tag="Robot/Data/LastPos/@X" />
      <ELEMENT Tag="Robot/Data/LastPos/@Y" />
      <ELEMENT Tag="Robot/Data/LastPos/@Z" />
      <ELEMENT Tag="Robot/Data/LastPos/@A" />
      <ELEMENT Tag="Robot/Data/LastPos/@B" />
      <ELEMENT Tag="Robot/Data/LastPos/@C" />
      <ELEMENT Tag="Robot/Data/ActPos/@X" />
      <ELEMENT Tag="Robot/Status" />
      <ELEMENT Tag="Robot/Mode" />
      <ELEMENT Tag="Robot/RobotLamp/GrenLamp/LightOn" />
    </XML>
  </SEND>
</ETHERNETKRL>
```

Programme

```

1 DEF XmlTransmit( )
2 Declaration
3 Communicated data
4 INI
5 Initialize sample data
6
7 RET=EKI_Init("XmlTransmit")
8 RET=EKI_Open("XmlTransmit")
9
10 Write data to connection
11 Send data to external program
12 Get received sensor data
13
14 RET=EKI_Close("XmlTransmit")
15 RET=EKI_Clear("XmlTransmit")
16
17 END

```

Ligne	Description
5	Initialisation des variables KRL par affectation de valeurs
7	EKI_Init() initialise le canal avec lequel l'EKI se connecte avec le système externe.
8	EKI_Open() ouvre le canal et se connecte avec le système externe.
10	Déclare des données dans le document XML sauvegardé pour l'envoi de données.
11	Envoie le document XML écrit au système externe.
12	Lit les données de capteur reçues dans la mémoire.
14	EKI_Close() ferme le canal.
15	EKI_Clear() efface le canal.

7.6 Exemple « XmlServer »

EKI est configurée en tant que serveur. Tant qu'il y a une connexion avec le système externe, \$FLAG[1] est activé.



Lorsque l'EKI est configuré en tant que serveur, le programme de serveur ne peut pas être utilisé sur le système externe. Un client simple peut être réalisé avec l'HyperTerminal Windows.

Fichier XML

```

<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <TYPE>Client</TYPE>
    </EXTERNAL>
    <INTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>54600</PORT>
      <ALIVE Set_Flag="1" />
    </INTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <XML>
      <ELEMENT Tag="Sensor/A" Type="BOOL" />
    </XML>
  </RECEIVE>
  <SEND>

```

```

<XML>
  <ELEMENT Tag="Robot/B" />
</XML>
</SEND>
</ETHERNETKRL>

```

Programme

```

1 DEF XmlServer( )
2 Declaration
3 INI
4
5 RET=EKI_Init("XmlServer")
6 RET=EKI_Open("XmlServer")
7
8 ; wait until server is connected
9 wait for $FLAG[1]
10 ; wait until server is disconnected
11 wait for $FLAG[1]==FALSE
12
13 RET=EKI_Clear("XmlServer")
14 END

```

Ligne	Description
5	EKI_Init() initialise le canal avec lequel le système externe se connecte avec l'EKI.
6	EKI_Open() ouvre le canal.
9	Lorsque le client externe s'est connecté avec succès avec le serveur, \$FLAG[1] est activé.
11	L'EKI étant configuré en tant que serveur, le contrôleur de robot que le canal soit fermé par le client externe. Si ceci est le cas, \$FLAG[1] est effacé.
13	EKI_Clear() efface le canal.

7.7 Exemple « XmlCallback »



Pour la communication avec le contrôleur de robot, il est nécessaire de régler les données d'exemple correspondantes dans le programme du serveur, ici, « Xml ».

EKI est configurée en tant que client. Des données de robot sont envoyées, des données de capteur sont reçues puis \$FLAG[1] est attendu. \$FLAG[1] signale que les données de capteur ont été lues.

Dans le fichier XML, il est configuré que \$FLAG[998] est activé lorsqu'EKI a reçu toutes les données de capteur. Ce drapeau déclenche l'interruption dans le programme. La configuration du tag « Capteur » en tant que tag d'évènement garantit que les données de capteur ne seront cherchées qu'une fois que toutes les données se trouvent dans les mémoires.

Lorsque les données de capteur sont lues, \$FLAG[998] est remis à zéro et \$FLAG[1] est activé.

Fichier XML

```

<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>59152</PORT>
    </EXTERNAL>
  </CONFIGURATION>
  <RECEIVE>

```

```

<XML>
  <ELEMENT Tag="Sensor/Message" Type="STRING" />
  <ELEMENT Tag="Sensor/Positions/Current/@X" Type="REAL" />
  <ELEMENT Tag="Sensor/Positions/Before/X" Type="REAL" />
  <ELEMENT Tag="Sensor/Nmb" Type="INT" />
  <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  <ELEMENT Tag="Sensor/Read/xyzabc" Type="FRAME" />
  <ELEMENT Tag="Sensor/Show/@error" Type="BOOL" />
  <ELEMENT Tag="Sensor/Show/@temp" Type="INT" />
  <ELEMENT Tag="Sensor/Show" Type="STRING" />
  <ELEMENT Tag="Sensor/Free" Type="INT" Set_Out="998" />
  <ELEMENT Tag="Sensor" Set_Flag="998" />
</XML>
</RECEIVE>
<SEND>
  <XML>
    <ELEMENT Tag="Robot/Data/LastPos/@X" />
    <ELEMENT Tag="Robot/Data/LastPos/@Y" />
    <ELEMENT Tag="Robot/Data/LastPos/@Z" />
    <ELEMENT Tag="Robot/Data/LastPos/@A" />
    <ELEMENT Tag="Robot/Data/LastPos/@B" />
    <ELEMENT Tag="Robot/Data/LastPos/@C" />
    <ELEMENT Tag="Robot/Data/ActPos/@X" />
    <ELEMENT Tag="Robot/Status" />
    <ELEMENT Tag="Robot/Mode" />
    <ELEMENT Tag="Robot/RobotLamp/GrenLamp/LightOn" />
  </XML>
</SEND>
</ETHERNETKRL>

```

Programme

```

1 DEF XmlCallBack( )
2 Declaration
3 Communicated data
4 INI
5 Define callback
6
7 RET=EKI_Init("XmlCallBack")
8 RET=EKI_Open("XmlCallBack")
9
10 Write data to connection
11 RET=EKI_Send("XmlCallBack","Robot")
12
13 ;wait until data read
14 WAIT FOR $FLAG[1]
15
16 RET=EKI_Close("XmlCallBack")
17 RET=EKI_Clear("XmlCallBack")
18 END
19
20 DEF GET_DATA()
21 Declaration
22 Initialize sample data
23 Get received sensor data
24 Signal read

```

Ligne	Description
5	Déclaration et activation de l'interruption
7	EKI_Init() initialise le canal avec lequel l'EKI se connecte avec le système externe.

Ligne	Description
8	EKI_Open() ouvre le canal.
10	Déclare des données dans le document XML sauvegardé pour l'envoi de données.
11	Envoie les données.
14	Attend \$FLAG[1]. Le drapeau d'évènement indique que toutes les données ont été lues.
16	EKI_Close() ferme le canal.
17	EKI_Clear() efface le canal.
20 ... 24	Initialisation des variables KRL par affectation de valeurs et lecture des données Lorsque toutes les données sont lues, \$FLAG[1] est activé.

Envoi de données

Des données de robot sont écrites par le programme KRL dans le document XML qui est ensuite envoyé par l'EKI au système externe.

```
<Robot>
  <Data>
    <LastPos X="..." Y="..." Z="..." A="..." B="..." C="...">
  </LastPos>
    <ActPos X="1000.12">
  </ActPos>
  </Data>
  <Status>12345678</Status>
  <Mode>ConnectSensor</Mode>
  <RobotLamp>
    <GrenLamp>
      <LightOn>1</LightOn>
    </GrenLamp>
  </RobotLamp>
</Robot>
```

Réception de données

Des données de capteur sont écrites par le programme de serveur dans le document XML qui est ensuite reçu par l'EKI.

```
<Sensor>
  <Message>Example message</Message>
  <Positions>
    <Current X="4645.2" />
    <Before>
      <X>0.9842</X>
    </Before>
  </Positions>
  <Nmb>8</Nmb>
  <Status>
    <IsActive>1</IsActive>
  </Status>
  <Read>
    <xyzabc X="210.3" Y="825.3" Z="234.3" A="84.2" B="12.3"
      C="43.5" />
  </Read>
  <Show error="0" temp="9929">Taginfo in attributes</Show>
  <Free>2912</Free>
</Sensor>
```


8 Diagnostic

8.1 Affichage des données de diagnostic

Condition préalable

- Droits d'utilisateurs : groupe de fonctions **Fonctions de diagnostic**

Procédure

1. Dans le menu principal, sélectionner **Diagnostic > Ecran de diagnostic**.
2. Dans le champ **Module**, sélectionner le module **EKI (EthernetKRL)**.

Description

Nom	Description
Mémoire totale	Mémoire totale disponible (octets)
Mémoire utilisée	Mémoire utilisée (octets)
Connexions du programme de robot	Nombre de connexions initialisées par l'interpréteur robot
Connexions du programme Submit	Nombre de connexions initialisées par l'interpréteur Submit
Connexions du système	Nombre de connexions initialisées par le système
Connexions Ethernet	Nombre de connexions ouvertes
Durée de traitement	Durée maximum nécessitée pour le traitement de données reçues (actualisation toutes les 5 secondes)
Messages d'avertissement	Nombre de messages d'avertissement
Messages de défaut	Nombre de messages de défauts



Les messages d'avertissement et de défauts sont également comptés lorsque l'émission automatique de messages et le protocole des messages ont été désactivés.

9 Messages

9.1 Protocole de défauts (table de messages EKI)

Tous les messages de défaut de l'EKI sont consignés par défaut dans un fichier LOG sous C:\KRC\ROBOTER\LOG\EthernetKRL.

9.2 Informations concernant les messages

Si un défaut s'est produit lors de l'appel d'une fonction ou lors de la réception de données, KUKA.Ethernet KRL renvoie le numéro de défaut. Un texte de message affiché sur la smartHMI est affecté aux numéros de défauts. Si l'émission automatique de messages est désactivée, le message peut continuer à être affiché sur la smartHMI avec EKI_CHECK().

Le chapitre "Messages" contient des messages sélectionnés. Il ne se consacre pas à tous les messages affichés dans la fenêtre de messages.

9.3 Messages de système du module: EthernetKRL (EKI)

9.3.1 EKI00002

Code de message	■ EKI00002
Texte du message	■ La mémoire système est épuisée
Type de message	■ Message de défaut
	
Effet	■ ARRÊT D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: Mémoire réservée insuffisante (>>> Page 61) Solution: augmenter la mémoire (>>> Page 62) ■ Cause: la configuration de connexion utilise trop de mémoire (>>> Page 62) Solution: adapter la configuration de connexion afin d'utiliser moins de mémoire (>>> Page 62) ■ Cause: plusieurs connexions avec un flux élevé de données sont actives (>>> Page 62) Solution: adapter la programmation afin d'utiliser moins de mémoire (>>> Page 63)

Cause: Mémoire réservée insuffisante

Description	La mémoire réservée pour Ethernet KRL est insuffisante.
Instruction de contrôle	■ Vérifier si la mémoire nécessaire peut être réduite par une autre configuration ou programmation. Si cela n'est pas possible, la mémoire ne peut être augmentée qu'après consultation de KUKA Roboter GmbH.

Solution: augmenter la mémoire

Une augmentation de la mémoire est seulement possible après consultation de KUKA Roboter GmbH. (>>> 11 "SAV KUKA" Page 115)

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Dans le répertoire C:\KRC\ROBOTER\Config\User\Common, ouvrir le fichier EthernetKRL.XML.
2. Inscrire la capacité de mémoire souhaitée en octets dans le paragraphe <EthernetKRL, dans l'élément <MemSize>.

```
<EthernetKRL>
  <Interface>
    <MemSize>1048576</MemSize>
  ...
</EthernetKRL>
```

3. Fermer le fichier et répondre par **Oui** à la question de sécurité demandant si la modification doit être sauvegardée.
4. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

Cause: la configuration de connexion utilise trop de mémoire**Description**

Les connexions Ethernet configurées nécessitent trop de mémoire.

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

Instruction de contrôle

- Contrôler le(s) fichier(s) XML afin de voir si les connexions Ethernet peuvent être configurées de façon à utiliser moins de mémoire.

Solution: adapter la configuration de connexion afin d'utiliser moins de mémoire**Condition préalable**

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

Cause: plusieurs connexions avec un flux élevé de données sont actives**Description**

Lorsque plusieurs connexions avec un flux élevé de données sont actives simultanément, cela peut entraîner une utilisation trop élevée de mémoire.

Instruction de contrôle

- Contrôler la programmation afin de voir si elle peut être modifiée de façon à utiliser moins de mémoire.

Solution: adapter la programmation afin d'utiliser moins de mémoire

Description La programmation doit être adaptée de façon à utiliser moins de mémoire.

9.3.2 EKI00003

Code de message	■ EKI00003
Texte du message	■ Échec de l'accès au fichier
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: fichier XML avec la configuration de connexion inexistant (>>> Page 63) Solution: restaurer le fichier XML contenant la configuration de connexion (>>> Page 63) ■ Cause: fichier XML contenant la configuration de connexion illisible (>>> Page 64) Solution: restaurer le fichier XML contenant la configuration de connexion (>>> Page 64)

Cause: fichier XML avec la configuration de connexion inexistant

Description La connexion Ethernet n'a pas été initialisée car aucun fichier XML portant le nom indiqué dans la fonction EKI_Init() n'est sauvegardé.

Fichier de configuration Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML —> EKI_INIT("EXT")

On peut vérifier si le fichier XML existe de la façon suivante :

Condition préalable

- Groupe d'utilisateur "Expert"
- Un programme est sélectionné ou ouvert.

Instruction de contrôle

1. Noter le nom de fichier utilisé dans la fonction EKI_Init().

```
RET = EKI_Init (« Nom de fichier »)
```

 - *Nom de fichier* : nom du fichier XML avec la configuration de connexion
2. Ouvrir le répertoire dans lequel les fichiers XML sont sauvegardés et vérifier s'il s'y trouve un fichier XML portant le nom indiqué dans la fonction EKI_Init().

Solution: restaurer le fichier XML contenant la configuration de connexion

Description Le fichier XML contenant la configuration de connexion doit être restauré et copié dans le répertoire C:\KRC\ROBOTER\Config\User\Common\EthernetKRL.

Condition préalable	■ Groupe d'utilisateurs "Expert" ■ Mode T1, T2 ou AUT
Procédure	■ Copier le fichier XML contenant la configuration de connexion dans le répertoire prévu. ■ Redémarrer le contrôleur de robot avec les réglages Dém. à froid et Nouvelle lecture des fichiers.

Cause: fichier XML contenant la configuration de connexion illisible

Description La connexion Ethernet n'a pas été initialisée car le fichier XML indiqué dans la fonction EKI_Init() n'est pas lisible. Le fichier XML est endommagé et ne peut pas être ouvert.

Fichier de configuration Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

On peut vérifier si le fichier XML est endommagé de la façon suivante :

Condition préalable	■ Groupe d'utilisateur "Expert" ■ Un programme est sélectionné ou ouvert.
Instruction de contrôle	1. Noter le nom de fichier utilisé dans la fonction EKI_Init(). RET = EKI_Init (« <i>Nom de fichier</i> ») ■ <i>Nom du fichier</i> : nom du fichier XML contenant la configuration de connexion 2. Ouvrir le répertoire dans lequel les fichiers XML sont sauvegardés et vérifier si le fichier XML peut y être ouvert.

Solution: restaurer le fichier XML contenant la configuration de connexion

Description Le fichier XML contenant la configuration de connexion doit être restauré et copié dans le répertoire C:\KRC\ROBOTER\Config\User\Common\EthernetKRL.

Condition préalable	■ Groupe d'utilisateurs "Expert" ■ Mode T1, T2 ou AUT
Procédure	■ Copier le fichier XML contenant la configuration de connexion dans le répertoire prévu. ■ Redémarrer le contrôleur de robot avec les réglages Dém. à froid et Nouvelle lecture des fichiers.

9.3.3 EKI00006

Code de message	■ EKI00006
Texte du message	■ Échec de l'interprétation de la configuration
Type de message	■ Message de défaut
	
Effet	■ ARRÊT D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.

Cause(s) possible(s)

- **Cause:** Erreur de schéma dans le fichier XML avec la configuration de connexion (>>> Page 65)

Solution: éliminer le défaut dans le fichier XML (>>> Page 65)

Cause: Erreur de schéma dans le fichier XML avec la configuration de connexion

Description

Le fichier XML avec la configuration de connexion ne peut pas être lu à cause d'une erreur de schéma.

KUKA.Ethernet KRL utilise le schéma XPath. La syntaxe définissant le schéma doit être respectée avec précision. Il ne doit p. ex. manquer aucun signe de ponctuation ou élément de structure.

L'écriture des éléments et des attributs dans le fichier XML ainsi que les majuscules et/ou minuscules sont prédéfinis et doivent être respectés.



Pour tout complément d'information concernant le schéma XPath, veuillez consulter la documentation KUKA.Ethernet KRL.

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

Solution: éliminer le défaut dans le fichier XML

Description

Les défauts dans le fichier XML doivent être éliminés.

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

9.3.4 EKI00007

Code de message

- EKI00007

Texte du message

- Échec de l'écriture des données pour l'envoi

Type de message

- Message de défaut



Effet

- ARRET D'URGENCE conforme à la trajectoire
- La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.

Cause(s) possible(s)

- **Cause:** structure XML pour l'envoi de données mal structurée (>>> Page 66)

Solution: structurer la structure XML en fonction des documents XML à envoyer (>>> Page 66)

Cause: structure XML pour l'envoi de données mal structurée**Description**

La structure XML pour l'envoi de données ne peut pas être écrite car celle-ci est conçue selon un schéma différent de celui du fichier XML à envoyer. KUKA.Ethernet KRL utilise le schéma XPath.



Pour tout complément d'information concernant le schéma XPath, veuillez consulter la documentation KUKA.Ethernet KRL.

Solution: structurer la structure XML en fonction des documents XML à envoyer**Description**

La structure XML pour l'envoi de données doit être structurée en fonction des documents XML à envoyer selon le schéma XPath.

9.3.5 EKI00009

Code de message	■ EKI00009
Texte du message	■ Liaison inexistante.
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: EKI_Init() n'est pas programmée ou est mal programmée (>>> Page 66) Solution: programmer la fonction correctement (>>> Page 67)

Cause: EKI_Init() n'est pas programmée ou est mal programmée**Description**

La connexion Ethernet n'a pas été initialisée car la fonction EKI_Init() n'est pas programmée ou est mal programmée.

Une connexion doit toujours être créée et initialisée avec la fonction EKI_Init(). Le fichier XML contenant la configuration de connexion indiqué dans la fonction est lu ce faisant.

RET = EKI_Init(CHAR[])

Fonction	Initialise un canal pour la communication Ethernet Les actions suivantes sont exécutées : ■ Lecture de la configuration de connexion ■ Réglage des mémoires de données ■ Préparation de la connexion Ethernet
Paramètre	Type: CHAR Nom du canal (nom du fichier XML contenant la configuration de connexion)
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_Init("Channel_1")

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

On peut vérifier si la fonction est programmée correctement de la façon suivante :

Condition préalable

- Groupe d'utilisateur "Expert"
- Un programme est sélectionné ou ouvert.

Instruction de contrôle

1. Vérifier si la ligne suivante est programmée :
`RET = EKI_Init (« Nom de fichier »)`
 - *Nom du fichier* : nom du fichier XML contenant la configuration de connexion
2. Vérifier si le nom de fichier indiqué dans la fonction EKI_Init() correspond au nom du fichier XML contenant la configuration de connexion.

Solution: programmer la fonction correctement**Description**

La fonction doit être programmée correctement.

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Marquer le programme dans le navigateur et appuyer sur **Ouvrir**. Le programme est affiché dans l'éditeur.
2. Chercher l'endroit correspondant dans le programme et l'éditer.
3. Fermer le fichier et répondre par **Oui** à la question de sécurité demandant si la modification doit être sauvegardée.

9.3.6 EKI00010

Code de message	■ EKI00010
Texte du message	■ La liaison Ethernet est coupée
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: EKI_Open() n'est pas programmée ou est mal programmée (>>> Page 67) Solution: programmer la fonction correctement (>>> Page 68)

Cause: EKI_Open() n'est pas programmée ou est mal programmée**Description**

La connexion Ethernet est initialisée mais n'est pas encore ouverte car la fonction EKI_Open() n'est pas programmée ou est mal programmée.

RET = EKI_Open(CHAR[])	
Fonction	Ouvre un canal initialisé Lorsqu'EKI est configurée en tant que client, l'EKI se connecte avec le système externe (= serveur). Lorsque l'EKI est configurée en tant que serveur, l'EKI attend la demande de connexion du système externe (= client).
Paramètre	Type: CHAR Nom du canal
RET	Type: EKI_STATUS Nom du canal (nom du fichier XML contenant la configuration de connexion)
Exemple	RET = EKI_Open("Channel_1")

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

On peut vérifier si la fonction est programmée correctement de la façon suivante :

Condition préalable

- Groupe d'utilisateur "Expert"
- Un programme est sélectionné ou ouvert.

Instruction de contrôle

1. Vérifier si la ligne suivante est programmée :
RET = EKI_Open (« Nom de fichier »)
 - Nom du fichier : nom du fichier XML contenant la configuration de connexion
2. Vérifier si le nom de fichier indiqué dans la fonction EKI_Open() correspond au nom du fichier utilisé dans la fonction EKI_Init().

Solution: programmer la fonction correctement

Description

La fonction doit être programmée correctement.

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Marquer le programme dans le navigateur et appuyer sur **Ouvrir**. Le programme est affiché dans l'éditeur.
2. Chercher l'endroit correspondant dans le programme et l'éditer.
3. Fermer le fichier et répondre par **Oui** à la question de sécurité demandant si la modification doit être sauvegardée.

9.3.7 EKI00011

Code de message
Texte du message
Type de message

- EKI00011
- La liaison Ethernet vers un système externe existe déjà
- Message de défaut



Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: connexion Ethernet déjà ouverte avec EKI_Open() (>>> Page 69) Solution: effacer la fonction contenant trop de programmations (>>> Page 69)

Cause: connexion Ethernet déjà ouverte avec EKI_Open()

Description

La connexion Ethernet a déjà été ouverte avec la fonction EKI_Open().

RET = EKI_Open(CHAR[])	
Fonction	Ouvre un canal initialisé Lorsqu'EKI est configurée en tant que client, l'EKI se connecte avec le système externe (= serveur). Lorsque l'EKI est configurée en tant que serveur, l'EKI attend la demande de connexion du système externe (= client).
Paramètre	Type: CHAR Nom du canal
RET	Type: EKI_STATUS Nom du canal (nom du fichier XML contenant la configuration de connexion)
Exemple	RET = EKI_Open("Channel_1")

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

On peut vérifier si la connexion est déjà ouverte de la façon suivante :

Condition préalable

- Groupe d'utilisateur "Expert"
- Un programme est sélectionné ou ouvert.

Instruction de contrôle

- Vérifier si la ligne suivante est programmée plusieurs fois entre l'initialisation et la fermeture de la connexion :
RET = EKI_Open (« Nom de fichier »)
■ *Nom de fichier* : nom du fichier XML contenant la configuration de connexion

Solution: effacer la fonction contenant trop de programmations

Description

La fonction contenant trop de programmations doit être effacée.

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Marquer le programme dans le navigateur et appuyer sur **Ouvrir**. Le programme est affiché dans l'éditeur.
2. Chercher l'endroit correspondant dans le programme et l'éditer.

3. Fermer le fichier et répondre par **Oui** à la question de sécurité demandant si la modification doit être sauvegardée.

9.3.8 EKI00012

Code de message	■ EKI00012
Texte du message	■ Échec de la création du serveur
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: L'adresse IP et/ou le numéro de port pour EKI ne sont pas indiqués ou mal indiqués. (>>> Page 70) Solution: saisir l'adresse IP et le numéro de port correctement dans le fichier XML (>>> Page 71)

Cause: L'adresse IP et/ou le numéro de port pour EKI ne sont pas indiqués ou mal indiqués.

Description EKI est configurée en tant que serveur et le système externe est configuré en tant que client. L'adresse IP et/ou le numéro de port pour l'EKI ne sont pas indiqués ou mal indiqués dans le fichier XML contenant la configuration de connexion. Afin de garantir la sécurité du réseau, il est possible de déterminer le sous-réseau à partir duquel on peut accéder à EKI.

Exemples Liaison uniquement possible avec le sous-réseau 192.168.X.X sur le port 54600 :

```
<INTERNAL>
<IP>192.168.23.1</IP>
<PORT>54600</PORT>
</INTERNAL>
```

Liaison possible avec tous les sous-réseaux configurés dans KLI sur le port 54600 :

```
<INTERNAL>
<IP>0.0.0.0</IP>
<PORT>54600</PORT>
</INTERNAL>
```

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

Instruction de contrôle

- Dans le fichier XML contenant la configuration de connexion, vérifier si l'adresse IP et le numéro de port sont indiqués correctement.

Solution: saisir l'adresse IP et le numéro de port correctement dans le fichier XML

Condition préalable	■ Groupe d'utilisateurs "Expert" ■ Mode T1, T2 ou AUT
Procédure	1. Modifier le fichier XML à votre convenance. 2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML. 3. Redémarrer le contrôleur de robot avec les réglages Dém. à froid et Nouvelle lecture des fichiers.

9.3.9 EKI00013

Code de message	■ EKI00013
Texte du message	■ Les paramètres Ethernet n'ont pas pu être initialisés
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: adresse IP et/ou le numéro de port pour le système externe non indiqués ou mal indiqués (>>> Page 71) Solution: saisir l'adresse IP et le numéro de port correctement dans le fichier XML (>>> Page 71)

Cause: adresse IP et/ou le numéro de port pour le système externe non indiqués ou mal indiqués

Description EKI est configurée en tant que client et le système externe est configuré en tant que serveur. L'adresse IP et/ou le numéro de port pour le système externe ne sont pas indiqués ou mal indiqués dans le fichier XML avec la configuration de connexion.

L'adresse IP et le numéro de port pour le système externe doivent être indiqués dans le paragraphe <EXTERNAL> ... </EXTERNAL> du fichier XML de la façon suivante :

- <IP>Adresse IP</IP>
- <PORT>Numéro de port</PORT>

Fichier de configuration Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

Instruction de contrôle ■ Dans le fichier XML contenant la configuration de connexion, vérifier si l'adresse IP et le numéro de port sont indiqués correctement.

Solution: saisir l'adresse IP et le numéro de port correctement dans le fichier XML

Condition préalable	■ Groupe d'utilisateurs "Expert" ■ Mode T1, T2 ou AUT
----------------------------	--

Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

9.3.10 EKI00014

Code de message	■ EKI00014
Texte du message	■ Aucune liaison Ethernet vers un système externe n'a pu être établie.
Type de message	■ Message de défaut
	
Effet	■ ARRÊT D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: adresse IP et/ou numéro de port incorrects indiqués (>>> Page 72) Solution: saisir l'adresse IP et le numéro de port correctement dans le fichier XML (>>> Page 73) ■ Cause: le câble réseau est défectueux ou n'est pas bien connecté (>>> Page 73) Solution: Remplacer un câble réseau ou le connecter correctement (>>> Page 73) ■ Cause: absence de connexion Ethernet à cause d'un défaut de logiciel, système externe (>>> Page 73) Solution: éliminer le défaut dans le logiciel du système externe (>>> Page 74) ■ Cause: absence de connexion Ethernet à cause d'un défaut de matériel, système externe (>>> Page 74) Solution: éliminer le défaut de matériel au système externe (>>> Page 74)

Cause: adresse IP et/ou numéro de port incorrects indiqués**Description**

Une adresse IP et/ou un numéro de port incorrects sont indiqués dans le fichier XML avec la configuration de connexion. Ils ne concordent pas avec le système externe ou l'EKI.

L'adresse IP et le numéro de port doivent être indiqués de la façon suivante, en fonction de la configuration du système externe en tant que serveur ou en tant que client :

- Paragraphe <EXTERNAL> ... </EXTERNAL>
 - <TYPE>Server</TYPE> ou <TYPE> non indiqué : le système externe est configuré en tant que serveur.
 - <TYPE>Client</TYPE> : le système externe est configuré en tant que client.
 - Lorsque TYPE = serveur, l'adresse IP et le numéro de port du système externe doivent être indiqués ici :

- <IP>Adresse IP</IP>
- <PORT>Numéro de port</PORT>

Lorsque TYPE = client, les paramètres de connexion indiqués ici sont ignorés.

- Paragraphe <INTERNAL> ... </INTERNAL>
 - Lorsque TYPE = client, l'adresse IP et le numéro de port de l'EKI doivent être indiqués ici :
 - <IP>Adresse IP</IP>
 - <PORT>Numéro de port</PORT>
- Lorsque TYPE = serveur, les paramètres de connexion indiqués ici sont ignorés.

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

Instruction de contrôle

- Dans le fichier XML contenant la configuration de connexion, vérifier si l'adresse IP et le numéro de port sont indiqués correctement.

Solution: saisir l'adresse IP et le numéro de port correctement dans le fichier XML

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

Cause: le câble réseau est défectueux ou n'est pas bien connecté

Description

Le câble réseau est défectueux ou la connexion est défectueuse.

On peut vérifier si le câble réseau et le connecteur sont connectés correctement de la façon suivante :

Instruction de contrôle

1. Contrôler toutes les connexions enfichables des câbles réseau et s'assurer de leur bonne fixation.
2. Permuter les câbles réseau.

Solution: Remplacer un câble réseau ou le connecter correctement

Procédure

- Remplacer un câble réseau ou le connecter correctement.

Cause: absence de connexion Ethernet à cause d'un défaut de logiciel, système externe

Description

Il n'y a pas de connexion Ethernet à cause d'un défaut dans le logiciel du système externe.

Instruction de contrôle

- Vérifier que le système externe est exempt d'erreurs de logiciel.

Solution: éliminer le défaut dans le logiciel du système externe

Description Le défaut dans le logiciel du système externe doit être éliminé.

Cause: absence de connexion Ethernet à cause d'un défaut de matériel, système externe

Description Il n'y a pas de connexion Ethernet à cause d'un défaut de matériel au système externe, provoqué p. ex. par un faux contact à une prise, une carte réseau défectueuse, etc.

Instruction de contrôle ■ Vérifier que le système externe est exempt de défauts de matériel.

Solution: éliminer le défaut de matériel au système externe

Description Le défaut de matériel au système externe doit être éliminé.

9.3.11 EKI00015

Code de message	■ EKI00015
Texte du message	■ Accès à une mémoire de réception vide
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: mémoire de données vide lors de l'accès avec EKI_Get...() (>>> Page 74) Solution: Modifier le programme (>>> Page 75)

Cause: mémoire de données vide lors de l'accès avec EKI_Get...()

Description Accès à une mémoire de données vide avec une fonction EKI_Get...().

L'accès à une mémoire vide peut être évité en interrogeant et évaluant la valeur renvoyée correspondante.

EKI_STATUS est la variable de structure globale dans laquelle les valeurs renvoyées de la fonction sont écrites. L'élément `Buff` d'EKI_STATUS est important pour l'évaluation.

`Buff` contient la valeur suivante :

- Nombre d'éléments se trouvant encore dans la mémoire après un accès.

Syntaxe GLOBAL STRUC EKI_STATUS INT `Buff`, `Read`, `Msg_No`, `BOOL Connected`, `INT Counter`

Exemple

```
...
REPEAT
    ret = EKI_GetInt("MyChannel", "Root/Number", value)
    DoSomething(value)
UNTIL (ret.Buff < 1)
...
```

Solution: Modifier le programme

Condition préalable	■ Groupe d'utilisateurs "Expert" ■ Mode T1, T2 ou AUT
Procédure	1. Marquer le programme dans le navigateur et appuyer sur Ouvrir. Le programme est affiché dans l'éditeur. 2. Chercher l'endroit correspondant dans le programme et l'éditer. 3. Fermer le fichier et répondre par Oui à la question de sécurité demandant si la modification doit être sauvegardée.

9.3.12 EKI00016

Code de message	■ EKI00016
Texte du message	■ L'élément n'a pas pu être trouvé.
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: élément dans la structure XML pour la réception de données non configuré ou mal configuré (>>> Page 75) Solution: éliminer le défaut dans le fichier XML (>>> Page 76) ■ Cause: nom d'élément mal programmé dans la fonction d'accès (>>> Page 76) Solution: programmer la fonction correctement (>>> Page 77)

Cause: élément dans la structure XML pour la réception de données non configuré ou mal configuré

Description	L'élément indiqué dans une fonction d'accès n'est pas configuré dans la structure XML pour la réception de données ou ne concorde pas avec l'élément configuré.
--------------------	---

Fonctions d'accès	Le paramètre 2 d'une fonction d'accès indique respectivement l'élément auquel on souhaite accéder.
--------------------------	--

Fonctions d'accès	
EKI_STATUS =	EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS =	EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
EKI_STATUS =	EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS =	EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS =	EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS =	EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS =	EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS =	EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS =	EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

La structure de réception est configurée dans le paragraphe <RECEIVE> ... </RECEIVE> du fichier XML. L'attribut Tag définit les éléments auxquels l'accès est possible.

Instruction de contrôle

1. Vérifier si l'élément sur lequel une tentative d'accès a été faite est bien configuré dans la structure de réception.
2. Vérifier si le nom d'élément programmé concorde avec le nom d'élément configuré.

Exemples**Lecture de données XML :**

Structure XML :

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmation :

```
RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Lecture d'un bloc de données binaires avec longueur fixe (10 octets) :

■ Données brutes :

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

■ Programmation :

```
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])
```

Solution: éliminer le défaut dans le fichier XML**Description**

Les défauts dans le fichier XML doivent être éliminés.

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

Cause: nom d'élément mal programmé dans la fonction d'accès**Description**

Le nom d'élément indiqué dans une fonction d'accès ne concorde pas avec le nom d'élément configuré dans la structure XML pour la réception de données.

Fonctions d'accès

Le paramètre 2 d'une fonction d'accès indique respectivement l'élément auquel on souhaite accéder.

Fonctions d'accès
EKI_STATUS = EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
EKI_STATUS = EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS = EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS = EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS = EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS = EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS = EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

La structure de réception est configurée dans le paragraphe <RECEIVE> ... </RECEIVE> du fichier XML. L'attribut Tag définit les éléments auxquels l'accès est possible.

Instruction de contrôle

- Vérifier si le nom d'élément programmé concorde avec le nom d'élément configuré.

Exemples

Lecture de données XML :

Structure XML :

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmation :

```
RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Lecture d'un bloc de données binaires avec longueur fixe (10 octets) :

- Données brutes :

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

- Programmation :

```
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])
```

Solution: programmer la fonction correctement

Description

La fonction doit être programmée correctement.

Condition préalable	■ Groupe d'utilisateurs "Expert" ■ Mode T1, T2 ou AUT
Procédure	1. Marquer le programme dans le navigateur et appuyer sur Ouvrir. Le programme est affiché dans l'éditeur. 2. Chercher l'endroit correspondant dans le programme et l'éditer. 3. Fermer le fichier et répondre par Oui à la question de sécurité demandant si la modification doit être sauvegardée.

9.3.13 EKI00017

Code de message	■ EKI00017
Texte du message	■ Échec de la préparation des données pour l'envoi
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: structure XML pour l'envoi de données mal configurée (>>> Page 78) Solution: éliminer le défaut dans le fichier XML (>>> Page 80) ■ Cause: EKI_Send() mal programmé (>>> Page 80) Solution: programmer la fonction correctement (>>> Page 82)

Cause: structure XML pour l'envoi de données mal configurée

Description	La structure XML pour l'envoi de données ne concorde pas avec le document XML sur lequel les données XML doivent être écrites. Elle ne concorde probablement pas non plus avec la programmation de la fonction EKI_Send().
--------------------	--

RET = EKI_Send(CHAR[], CHAR[], INT)	
Fonction	Envoie des données par un canal (>>> 6.2.4 "Envoi de données" Page 36)
Paramètre 1	Type: CHAR Nom du canal ouvert par lequel l'envoi doit être effectué

RET = EKI_Send(CHAR[], CHAR[], INT)	
Paramètre 2	Type: CHAR Définit l'étendue des données à envoyer. Pour une communication via structure XML : ■ Impression de XPath de l'élément à envoyer de la structure XML configurée pour l'envoi de données. Si seul l'élément racine est indiqué, l'ensemble de la structure XML est transféré (voir l'exemple 1). Si seule une partie de la structure XML doit être transférée, c'est-à-dire l'élément d'un niveau subordonné, il faut indiquer le chemin jusqu'à cet élément en partant de cet élément racine (voir l'exemple 2). ■ Alternative : chaîne de caractères quelconque de longueur variable devant être transmise Pour une communication via données brutes : ■ chaîne de caractères quelconque devant être transmise : ■ Pour des blocs de données binaires avec longueur fixe (attribut Type= « BYTE ») : chaîne de caractères quelconque de longueur fixe La longueur de la chaîne de caractères en octets doit correspondre exactement à l'attribut configuré Size. En cas de dépassement, un message de défaut est émis, si la longueur n'est pas atteinte, un message d'avertissement est émis. ■ Pour des blocs de données binaires avec séquence de suffixes variable (attribut Type= « STREAM ») : chaîne de caractères quelconque de longueur variable La séquence de suffixes est envoyée automatiquement. Remarque : si une chaîne de caractères quelconque de longueur variable contient un caractère ZÉRO ASCII, seule la partie concernant ce symbole est transférée. Si cela n'est pas souhaité, il faudra régler le paramètre 3 en conséquence.
Paramètre 3 (option)	Type: INT N'est important que pour l'envoi de chaînes de caractères quelconques dont la longueur est variable (voir le paramètre 2) : nombre maximum de caractères à envoyer Le caractère ZÉRO ASCII est ignoré. Si la chaîne de caractères est trop longue, celle-ci est coupée.
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple 1	RET = EKI_Send("Channel_1", "Root")
Exemple 2	RET = EKI_Send("Channel_1", "Root/Test")
Exemple 3	RET = EKI_Send("Channel_1", MyBytes[])
Exemple 4	RET = EKI_Send("Channel_1", MyBytes[], 6)

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

La structure d'envoi est configurée dans le paragraphe <SEND> ... </SEND> du fichier XML.

- Instruction de contrôle**
1. Contrôler la configuration de la structure d'envoi dans le fichier XML.
 2. Vérifier si la programmation des données envoyées concorde avec les données envoyées.

Solution: éliminer le défaut dans le fichier XML

Description Les défauts dans le fichier XML doivent être éliminés.

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

Cause: EKI_Send() mal programmé

Description Les données à envoyer sont mal indiquées dans la fonction EKI_Send(). Elles ne concordent probablement pas avec la structure XML configurée pour l'envoi de données.

RET = EKI_Send(CHAR[], CHAR[], INT)	
Fonction	Envoie des données par un canal (>>> 6.2.4 "Envoi de données" Page 36)
Paramètre 1	Type: CHAR Nom du canal ouvert par lequel l'envoi doit être effectué

RET = EKI_Send(CHAR[], CHAR[], INT)	
Paramètre 2	Type: CHAR Définit l'étendue des données à envoyer. Pour une communication via structure XML : ■ Impression de XPath de l'élément à envoyer de la structure XML configurée pour l'envoi de données. Si seul l'élément racine est indiqué, l'ensemble de la structure XML est transféré (voir l'exemple 1). Si seule une partie de la structure XML doit être transférée, c'est-à-dire l'élément d'un niveau subordonné, il faut indiquer le chemin jusqu'à cet élément en partant de cet élément racine (voir l'exemple 2). ■ Alternative : chaîne de caractères quelconque de longueur variable devant être transmise Pour une communication via données brutes : ■ chaîne de caractères quelconque devant être transmise : ■ Pour des blocs de données binaires avec longueur fixe (attribut Type= « BYTE ») : chaîne de caractères quelconque de longueur fixe La longueur de la chaîne de caractères en octets doit correspondre exactement à l'attribut configuré Size. En cas de dépassement, un message de défaut est émis, si la longueur n'est pas atteinte, un message d'avertissement est émis. ■ Pour des blocs de données binaires avec séquence de suffixes variable (attribut Type= « STREAM ») : chaîne de caractères quelconque de longueur variable La séquence de suffixes est envoyée automatiquement. Remarque : si une chaîne de caractères quelconque de longueur variable contient un caractère ZÉRO ASCII, seule la partie concernant ce symbole est transférée. Si cela n'est pas souhaité, il faudra régler le paramètre 3 en conséquence.
Paramètre 3 (option)	Type: INT N'est important que pour l'envoi de chaînes de caractères quelconques dont la longueur est variable (voir le paramètre 2) : nombre maximum de caractères à envoyer Le caractère ZÉRO ASCII est ignoré. Si la chaîne de caractères est trop longue, celle-ci est coupée.
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple 1	RET = EKI_Send("Channel_1", "Root")
Exemple 2	RET = EKI_Send("Channel_1", "Root/Test")
Exemple 3	RET = EKI_Send("Channel_1", MyBytes[])
Exemple 4	RET = EKI_Send("Channel_1", MyBytes[], 6)

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

La structure d'envoi est configurée dans le paragraphe <SEND> ... </SEND> du fichier XML.

Instruction de contrôle	1. Contrôler la configuration de la structure d'envoi dans le fichier XML.
	2. Vérifier si la programmation des données envoyées concorde avec les données envoyées.

Solution: programmer la fonction correctement

Description	La fonction doit être programmée correctement.
Condition préalable	■ Groupe d'utilisateurs "Expert"
	■ Mode T1, T2 ou AUT
Procédure	1. Marquer le programme dans le navigateur et appuyer sur Ouvrir . Le programme est affiché dans l'éditeur.
	2. Chercher l'endroit correspondant dans le programme et l'éditer.
	3. Fermer le fichier et répondre par Oui à la question de sécurité demandant si la modification doit être sauvegardée.

9.3.14 EKI00018

Code de message	■ EKI00018
Texte du message	■ Échec de l'envoi de données
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire
	■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: le câble réseau est défectueux ou n'est pas bien connecté (>>> Page 82) Solution: Remplacer un câble réseau ou le connecter correctement (>>> Page 83)
	■ Cause: absence de connexion Ethernet à cause d'un défaut de logiciel, système externe (>>> Page 83) Solution: éliminer le défaut dans le logiciel du système externe (>>> Page 83)
	■ Cause: absence de connexion Ethernet à cause d'un défaut de matériel, système externe (>>> Page 83) Solution: éliminer le défaut de matériel au système externe (>>> Page 83)

Cause: le câble réseau est défectueux ou n'est pas bien connecté

Description	Le câble réseau est défectueux ou la connexion est défectueuse. On peut vérifier si le câble réseau et le connecteur sont connectés correctement de la façon suivante :
Instruction de contrôle	1. Contrôler toutes les connexions enfichables des câbles réseau et s'assurer de leur bonne fixation.
	2. Permuter les câbles réseau.

Solution: Remplacer un câble réseau ou le connecter correctement

Procédure ■ Remplacer un câble réseau ou le connecter correctement.

Cause: absence de connexion Ethernet à cause d'un défaut de logiciel, système externe

Description Il n'y a pas de connexion Ethernet à cause d'un défaut dans le logiciel du système externe.

Instruction de contrôle ■ Vérifier que le système externe est exempt d'erreurs de logiciel.

Solution: éliminer le défaut dans le logiciel du système externe

Description Le défaut dans le logiciel du système externe doit être éliminé.

Cause: absence de connexion Ethernet à cause d'un défaut de matériel, système externe

Description Il n'y a pas de connexion Ethernet à cause d'un défaut de matériel au système externe, provoqué p. ex. par un faux contact à une prise, une carte réseau défectueuse, etc.

Instruction de contrôle ■ Vérifier que le système externe est exempt de défauts de matériel.

Solution: éliminer le défaut de matériel au système externe

Description Le défaut de matériel au système externe doit être éliminé.

9.3.15 EKI00019

Code de message	■ EKI00019
Texte du message	■ Il n'y a pas de données pour l'envoi
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: EKI_Send() ne contient aucune donnée à envoyer (>>> Page 83) Solution: programmer la fonction correctement (>>> Page 85)

Cause: EKI_Send() ne contient aucune donnée à envoyer

Description Une fonction d'envoi EKI_Send() ne contient aucune donnée à envoyer (paramètre 2).

RET = EKI_Send(CHAR[], CHAR[], INT)	
Fonction	Envoie des données par un canal (>>> 6.2.4 "Envoi de données" Page 36)
Paramètre 1	Type: CHAR Nom du canal ouvert par lequel l'envoi doit être effectué
Paramètre 2	Type: CHAR Défini l'étendue des données à envoyer. Pour une communication via structure XML : ■ Impression de XPath de l'élément à envoyer de la structure XML configurée pour l'envoi de données. Si seul l'élément racine est indiqué, l'ensemble de la structure XML est transféré (voir l'exemple 1). Si seule une partie de la structure XML doit être transférée, c'est-à-dire l'élément d'un niveau subordonné, il faut indiquer le chemin jusqu'à cet élément en partant de cet élément racine (voir l'exemple 2). ■ Alternative : chaîne de caractères quelconque de longueur variable devant être transmise Pour une communication via données brutes : ■ chaîne de caractères quelconque devant être transmise : ■ Pour des blocs de données binaires avec longueur fixe (attribut <code>Type= « BYTE »</code>) : chaîne de caractères quelconque de longueur fixe La longueur de la chaîne de caractères en octets doit correspondre exactement à l'attribut configuré <code>Size</code>. En cas de dépassement, un message de défaut est émis, si la longueur n'est pas atteinte, un message d'avertissement est émis. ■ Pour des blocs de données binaires avec séquence de suffixes variable (attribut <code>Type= « STREAM »</code>) : chaîne de caractères quelconque de longueur variable La séquence de suffixes est envoyée automatiquement. Remarque : si une chaîne de caractères quelconque de longueur variable contient un caractère ZÉRO ASCII, seule la partie concernant ce symbole est transférée. Si cela n'est pas souhaité, il faudra régler le paramètre 3 en conséquence.
Paramètre 3 (option)	Type: INT N'est important que pour l'envoi de chaînes de caractères quelconques dont la longueur est variable (voir le paramètre 2) : nombre maximum de caractères à envoyer Le caractère ZÉRO ASCII est ignoré. Si la chaîne de caractères est trop longue, celle-ci est coupée.
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple 1	RET = EKI_Send("Channel_1", "Root")
Exemple 2	RET = EKI_Send("Channel_1", "Root/Test")
Exemple 3	RET = EKI_Send("Channel_1", MyBytes[])
Exemple 4	RET = EKI_Send("Channel_1", MyBytes[], 6)

Instruction de contrôle

- Vérifier dans le programme KRL si les données à envoyer sont indiquées.

Solution: programmer la fonction correctement

Description	La fonction doit être programmée correctement.
Condition préalable	■ Groupe d'utilisateurs "Expert" ■ Mode T1, T2 ou AUT
Procédure	1. Marquer le programme dans le navigateur et appuyer sur Ouvrir. Le programme est affiché dans l'éditeur. 2. Chercher l'endroit correspondant dans le programme et l'éditer. 3. Fermer le fichier et répondre par Oui à la question de sécurité demandant si la modification doit être sauvegardée.

9.3.16 EKI00020

Code de message	■ EKI00020
Texte du message	■ Les types de données ne concordent pas
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: type de données dans la structure XML pour la réception de données mal configuré (>>> Page 85) Solution: éliminer le défaut dans le fichier XML (>>> Page 86) ■ Cause: type de données mal programmé dans la fonction d'accès (>>> Page 87) Solution: programmer la fonction correctement (>>> Page 88)

Cause: type de données dans la structure XML pour la réception de données mal configuré

Description	Le type de données indiqué dans une fonction d'accès ne concorde pas avec le type de données configuré dans la structure XML pour la réception de données.
Fonctions d'accès	Le paramètre 3 d'une fonction d'accès indique respectivement le type de données de l'élément auquel on souhaite accéder.

Fonctions d'accès
EKI_STATUS = EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
EKI_STATUS = EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS = EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS = EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS = EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS = EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS = EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])

Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

La structure de réception est configurée dans le paragraphe <RECEIVE> ... </RECEIVE> du fichier XML. L'attribut `Type` définit le nom de type d'un élément.

Instruction de contrôle

- Vérifier si le type de données programmé concorde avec le type de données de l'élément configuré dans la structure de réception.

Exemples**Lecture de données XML :**

Structure XML :

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmation :

```
...
CHAR valueChar[256]
BOOL valueBOOL
...
RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Lecture d'un bloc de données binaires avec longueur fixe (10 octets) :

- Données brutes :

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

- Programmation :

```
...
CHAR Bytes[10]
...
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])
```

Solution: éliminer le défaut dans le fichier XML**Description**

Les défauts dans le fichier XML doivent être éliminés.

Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

Cause: type de données mal programmé dans la fonction d'accès

Description Le type de données indiqué dans une fonction d'accès ne concorde pas avec le type de données configuré dans la structure XML pour la réception de données.

Fonctions d'accès Le paramètre 3 d'une fonction d'accès indique respectivement le type de données de l'élément auquel on souhaite accéder.

Fonctions d'accès
EKI_STATUS = EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
EKI_STATUS = EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS = EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS = EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS = EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS = EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS = EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])

Fichier de configuration Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	Exemple : ... \EXT.XML → EKI_INIT("EXT")

La structure de réception est configurée dans le paragraphe <RECEIVE> ... </RECEIVE> du fichier XML. L'attribut `Type` définit le nom de type d'un élément.

Instruction de contrôle ■ Vérifier si le type de données programmé concorde avec le type de données de l'élément configuré dans la structure de réception.

Exemples

Lecture de données XML :

Structure XML :

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmation :

```
...
CHAR valueChar[256]
BOOL valueBOOL
...
RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Lecture d'un bloc de données binaires avec longueur fixe (10 octets) :

■ Données brutes :

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

■ Programmation :

```
...
CHAR Bytes[10]
...
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])
```

Solution: programmer la fonction correctement

Description	La fonction doit être programmée correctement.
Condition préalable	■ Groupe d'utilisateurs "Expert" ■ Mode T1, T2 ou AUT
Procédure	1. Marquer le programme dans le navigateur et appuyer sur Ouvrir. Le programme est affiché dans l'éditeur. 2. Chercher l'endroit correspondant dans le programme et l'éditer. 3. Fermer le fichier et répondre par Oui à la question de sécurité demandant si la modification doit être sauvegardée.

9.3.17 EKI00021

Code de message	■ EKI00021
Texte du message	■ La mémoire du système n'est pas suffisante pour la gestion de données maximum
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ Cause: Mémoire réservée insuffisante (>>> Page 88) Solution: augmenter la mémoire (>>> Page 88) ■ Cause: la configuration de connexion utilise trop de mémoire (>>> Page 89) Solution: adapter la configuration de connexion afin d'utiliser moins de mémoire (>>> Page 89)

Cause: Mémoire réservée insuffisante

Description	La mémoire réservée pour Ethernet KRL est insuffisante.
Instruction de contrôle	■ Vérifier si la mémoire nécessaire peut être réduite par une autre configuration ou programmation. Si cela n'est pas possible, la mémoire ne peut être augmentée qu'après consultation de KUKA Roboter GmbH.

Solution: augmenter la mémoire

 Une augmentation de la mémoire est seulement possible après consultation de KUKA Roboter GmbH. (>>> 11 "SAV KUKA" Page 115)

- Condition préalable**
- Groupe d'utilisateurs "Expert"
 - Mode T1, T2 ou AUT
- Procédure**
1. Dans le répertoire C:\KRC\ROBOTER\Config\User\Common, ouvrir le fichier EthernetKRL.XML.
 2. Inscrire la capacité de mémoire souhaitée en octets dans le paragraphe <EthernetKRL, dans l'élément <MemSize>.
- ```
<EthernetKRL>
 <Interface>
 <MemSize>1048576</MemSize>
 ...
</EthernetKRL>
```
3. Fermer le fichier et répondre par **Oui** à la question de sécurité demandant si la modification doit être sauvegardée.
  4. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

#### Cause: la configuration de connexion utilise trop de mémoire

- Description** Les connexions Ethernet configurées nécessitent trop de mémoire.
- Fichier de configuration** Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	<b>Exemple :</b> ... \EXT.XML → EKI_INIT("EXT")

- Instruction de contrôle**
- Contrôler le(s) fichier(s) XML afin de voir si les connexions Ethernet peuvent être configurées de façon à utiliser moins de mémoire.

#### Solution: adapter la configuration de connexion afin d'utiliser moins de mémoire

- Condition préalable**
- Groupe d'utilisateurs "Expert"
  - Mode T1, T2 ou AUT
- Procédure**
1. Modifier le fichier XML à votre convenance.
  2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
  3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

#### 9.3.18 EKI00022

Code de message	■ EKI00022
Texte du message	■ Défaut lors de la lecture de la configuration. Erreur XML.
Type de message	■ Message de défaut
	
Effet	■ ARRÊT D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.

Cause(s) possible(s)

- **Cause:** Erreur de schéma dans le fichier XML avec la configuration de connexion (>>> Page 90)

**Solution:** éliminer le défaut dans le fichier XML (>>> Page 90)

### Cause: Erreur de schéma dans le fichier XML avec la configuration de connexion

#### Description

Le fichier XML avec la configuration de connexion ne peut pas être lu à cause d'une erreur de schéma.

KUKA.Ethernet KRL utilise le schéma XPath. La syntaxe définissant le schéma doit être respectée avec précision. Il ne doit p. ex. manquer aucun signe de ponctuation ou élément de structure.

L'écriture des éléments et des attributs dans le fichier XML ainsi que les majuscules et/ou minuscules sont prédéfinis et doivent être respectés.



Pour tout complément d'information concernant le schéma XPath, veuillez consulter la documentation KUKA.Ethernet KRL.

#### Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	<b>Exemple :</b> ...\EXT.XML → EKI_INIT("EXT")

### Solution: éliminer le défaut dans le fichier XML

#### Description

Les défauts dans le fichier XML doivent être éliminés.

#### Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

#### Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

### 9.3.19 EKI00023

Code de message

- EKI00023

Texte du message

- L'initialisation est déjà exécutée.

Type de message

- Message de défaut



Effet

- ARRÊT D'URGENCE conforme à la trajectoire
- La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.

Cause(s) possible(s)

- **Cause:** connexion Ethernet déjà initialisée avec EKI\_Init() (>>> Page 91)

**Solution:** effacer la fonction contenant trop de programmations (>>> Page 91)

**Cause: connexion Ethernet déjà initialisée avec EKI\_Init()**

**Description** La connexion Ethernet a déjà été initialisée avec la fonction EKI\_Init().

<b>RET = EKI_Init(CHAR[])</b>	
Fonction	Initialise un canal pour la communication Ethernet Les actions suivantes sont exécutées :      ■ Lecture de la configuration de connexion     ■ Réglage des mémoires de données     ■ Préparation de la connexion Ethernet
Paramètre	Type: CHAR Nom du canal (nom du fichier XML contenant la configuration de connexion)
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_Init("Channel_1")

**On peut vérifier si la connexion est déjà initialisée de la façon suivante :**

- Condition préalable**
- Groupe d'utilisateur "Expert"
  - Un programme est sélectionné ou ouvert.
- Instruction de contrôle**
- Vérifier si la ligne suivante est programmée plusieurs fois entre la première initialisation et la fermeture de la connexion :  

```
RET = EKI_Init (« Nom de fichier »)
```

    - *Nom du fichier* : nom du fichier XML contenant la configuration de connexion

**Solution: effacer la fonction contenant trop de programmations**

**Description** La fonction contenant trop de programmations doit être effacée.

- Condition préalable**
- Groupe d'utilisateurs "Expert"
  - Mode T1, T2 ou AUT

- Procédure**
1. Marquer le programme dans le navigateur et appuyer sur **Ouvrir**. Le programme est affiché dans l'éditeur.
  2. Chercher l'endroit correspondant dans le programme et l'éditer.
  3. Fermer le fichier et répondre par **Oui** à la question de sécurité demandant si la modification doit être sauvegardée.

**9.3.20 EKI00024**

Code de message	■ EKI00024
Texte du message	■ Échec de la liaison à des paramètres internes (port, IP)
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.

Cause(s) possible(s)

- **Cause:** L'adresse IP et/ou le numéro de port pour EKI ne sont pas indiqués ou mal indiqués. (>>> Page 92)

**Solution:** saisir l'adresse IP et le numéro de port correctement dans le fichier XML (>>> Page 92)

**Cause:** L'adresse IP et/ou le numéro de port pour EKI ne sont pas indiqués ou mal indiqués.

#### Description

EKI est configurée en tant que serveur et le système externe est configuré en tant que client. L'adresse IP et/ou le numéro de port pour l'EKI ne sont pas indiqués ou mal indiqués dans le fichier XML contenant la configuration de connexion. Afin de garantir la sécurité du réseau, il est possible de déterminer le sous-réseau à partir duquel on peut accéder à EKI.

#### Exemples

Liaison uniquement possible avec le sous-réseau 192.168.X.X sur le port 54600 :

```
<INTERNAL>
<IP>192.168.23.1</IP>
<PORT>54600</PORT>
</INTERNAL>
```

Liaison possible avec tous les sous-réseaux configurés dans KLI sur le port 54600 :

```
<INTERNAL>
<IP>0.0.0.0</IP>
<PORT>54600</PORT>
</INTERNAL>
```

#### Fichier de configuration

Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	<b>Exemple :</b> ... \EXT.XML —> EKI_INIT("EXT")

#### Instruction de contrôle

- Dans le fichier XML contenant la configuration de connexion, vérifier si l'adresse IP et le numéro de port sont indiqués correctement.

**Solution:** saisir l'adresse IP et le numéro de port correctement dans le fichier XML

#### Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

#### Procédure

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

## 9.3.21 EKI00027

Code de message	■ EKI00027
Texte du message	■ Le champ KRL CHAR[] est trop petit.
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ <b>Cause:</b> Champ CHAR trop petit pour les données reçues (&gt;&gt;&gt; Page 93)     ■ <b>Solution:</b> augmenter la taille de la mémoire du champ CHAR (&gt;&gt;&gt; Page 93)

**Cause: Champ CHAR trop petit pour les données reçues**

<b>Description</b>	Le champ CHAR défini dans le programme KRL est trop petit pour les données reçues.
<b>Instruction de contrôle</b>	■ Contrôler la longueur maximum acceptée pour les blocs de données envoyés par le système externe.

**Solution: augmenter la taille de la mémoire du champ CHAR**

<b>Description</b>	La taille de la mémoire du champ CHAR doit être augmentée.
<b>Condition préalable</b>	■ Groupe d'utilisateurs "Expert"     ■ Mode T1, T2 ou AUT
<b>Procédure</b>	1. Marquer le programme dans le navigateur et appuyer sur <b>Ouvrir</b>. Le programme est affiché dans l'éditeur.     2. Chercher l'endroit correspondant dans le programme et l'éditer.     3. Fermer le fichier et répondre par <b>Oui</b> à la question de sécurité demandant si la modification doit être sauvegardée.

## 9.3.22 EKI00512

Code de message	■ EKI00512
Texte du message	■ Liaison Ethernet perturbée
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ <b>Cause:</b> le câble réseau est défectueux ou n'est pas bien connecté (&gt;&gt;&gt; Page 94)     ■ <b>Solution:</b> Remplacer un câble réseau ou le connecter correctement (&gt;&gt;&gt; Page 94)

- **Cause:** absence de connexion Ethernet à cause d'un défaut de logiciel, système externe (>>> Page 94)  
**Solution:** éliminer le défaut dans le logiciel du système externe (>>> Page 94)
- **Cause:** absence de connexion Ethernet à cause d'un défaut de matériel, système externe (>>> Page 94)  
**Solution:** éliminer le défaut de matériel au système externe (>>> Page 94)

**Cause: le câble réseau est défectueux ou n'est pas bien connecté**

<b>Description</b>	Le câble réseau est défectueux ou la connexion est défectueuse. <b>On peut vérifier si le câble réseau et le connecteur sont connectés correctement de la façon suivante :</b>
<b>Instruction de contrôle</b>	1. Contrôler toutes les connexions enfichables des câbles réseau et s'assurer de leur bonne fixation.     2. Permuter les câbles réseau.

**Solution: Remplacer un câble réseau ou le connecter correctement**

<b>Procédure</b>	■ Remplacer un câble réseau ou le connecter correctement.
------------------	-----------------------------------------------------------

**Cause: absence de connexion Ethernet à cause d'un défaut de logiciel, système externe**

<b>Description</b>	Il n'y a pas de connexion Ethernet à cause d'un défaut dans le logiciel du système externe.
<b>Instruction de contrôle</b>	■ Vérifier que le système externe est exempt d'erreurs de logiciel.

**Solution: éliminer le défaut dans le logiciel du système externe**

<b>Description</b>	Le défaut dans le logiciel du système externe doit être éliminé.
--------------------	------------------------------------------------------------------

**Cause: absence de connexion Ethernet à cause d'un défaut de matériel, système externe**

<b>Description</b>	Il n'y a pas de connexion Ethernet à cause d'un défaut de matériel au système externe, provoqué p. ex. par un faux contact à une prise, une carte réseau défectueuse, etc.
<b>Instruction de contrôle</b>	■ Vérifier que le système externe est exempt de défauts de matériel.

**Solution: éliminer le défaut de matériel au système externe**

<b>Description</b>	Le défaut de matériel au système externe doit être éliminé.
--------------------	-------------------------------------------------------------

## 9.3.23 EKI00768

Code de message	■ EKI00768
Texte du message	■ Le Ping ne signale aucun contact
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ <b>Cause:</b> le câble réseau est défectueux ou n'est pas bien connecté (&gt;&gt;&gt; Page 95) <b>Solution:</b> Remplacer un câble réseau ou le connecter correctement (&gt;&gt;&gt; Page 95)     ■ <b>Cause:</b> absence de connexion Ethernet à cause d'un défaut de matériel, système externe (&gt;&gt;&gt; Page 95) <b>Solution:</b> éliminer le défaut de matériel au système externe (&gt;&gt;&gt; Page 95)

**Cause: le câble réseau est défectueux ou n'est pas bien connecté**

<b>Description</b>	Le câble réseau est défectueux ou la connexion est défectueuse. <b>On peut vérifier si le câble réseau et le connecteur sont connectés correctement de la façon suivante :</b>
<b>Instruction de contrôle</b>	1. Contrôler toutes les connexions enfichables des câbles réseau et s'assurer de leur bonne fixation.     2. Permuter les câbles réseau.

**Solution: Remplacer un câble réseau ou le connecter correctement**

<b>Procédure</b>	■ Remplacer un câble réseau ou le connecter correctement.
------------------	-----------------------------------------------------------

**Cause: absence de connexion Ethernet à cause d'un défaut de matériel, système externe**

<b>Description</b>	Il n'y a pas de connexion Ethernet à cause d'un défaut de matériel au système externe, provoqué p. ex. par un faux contact à une prise, une carte réseau défectueuse, etc.
<b>Instruction de contrôle</b>	■ Vérifier que le système externe est exempt de défauts de matériel.

**Solution: éliminer le défaut de matériel au système externe**

<b>Description</b>	Le défaut de matériel au système externe doit être éliminé.
--------------------	-------------------------------------------------------------

## 9.3.24 EKI01024

Code de message	■ EKI01024
Texte du message	■ Défaut lors de la lecture des données XML reçues

Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ <b>Cause:</b> le document XML envoyé ne correspond pas au schéma XPath (&gt;&gt;&gt; Page 96)     <b>Solution:</b> structurer le document XML envoyé selon le schéma XPath (&gt;&gt;&gt; Page 96)

### Cause: le document XML envoyé ne correspond pas au schéma XPath

**Description** Le document XML envoyé par le système externe ne correspond pas au schéma XPath.



Pour tout complément d'information concernant le schéma XPath, veuillez consulter la documentation KUKA.Ethernet KRL.

### Solution: structurer le document XML envoyé selon le schéma XPath

**Description** Le document XML envoyé par le système externe doit être structuré selon le schéma XPath.

#### 9.3.25 EKI01280

Code de message	■ EKI01280
Texte du message	■ La limite d'éléments pouvant être sauvegardés est atteinte
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ <b>Cause:</b> limite de mémoire atteinte (nombre l'éléments de données par mémoire) (&gt;&gt;&gt; Page 97)     <b>Solution:</b> augmenter la limite de mémoire dans le fichier XML (&gt;&gt;&gt; Page 97)     ■ <b>Cause:</b> limite de mémoire atteinte (nombre l'éléments de données par mémoire) (&gt;&gt;&gt; Page 97)     <b>Solution:</b> contrôler la mémoire de données de façon cyclique et la verrouiller avant que la limite soit atteinte (&gt;&gt;&gt; Page 98)     ■ <b>Cause:</b> EKI : la mémoire EKI n'est pas lue (&gt;&gt;&gt; Page 98)     <b>Solution:</b> Modifier le programme (&gt;&gt;&gt; Page 99)

**Cause: limite de mémoire atteinte (nombre l'éléments de données par mémoire)**

**Description** Le nombre d'éléments de données pouvant être sauvegardés est limité. Cette limite de mémoire a été atteinte. La connexion Ethernet a été fermée afin d'éviter la réception de données supplémentaires.

Le nombre maximum d'éléments pouvant être sauvegardés est défini dans le fichier XML contenant la configuration de connexion :

- Paragraphe <INTERNAL> ... </INTERNAL>
- Élément <BUFFERING ... Limit="..."/>
  - Nombre maximum possible : 512
- Si l'élément BUFFERING n'est pas configuré, la valeur par défaut est variable :
  - <BUFFERING Mode="FIFO" Limit="16"/>

**Fichier de configuration** Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	<b>Exemple</b> : ... \EXT.XML —> EKI_INIT("EXT")

**Instruction de contrôle** ■ Dans le fichier XML contenant la configuration de connexion, vérifier quelle est la valeur limite du nombre d'éléments pouvant être sauvegardés.

**Solution: augmenter la limite de mémoire dans le fichier XML**

**Description** La limite de mémoire peut être augmentée dans le fichier XML.

**Condition préalable** ■ Groupe d'utilisateurs "Expert"  
 ■ Mode T1, T2 ou AUT

**Procédure**

1. Modifier le fichier XML à votre convenance.
2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.
3. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

**Cause: limite de mémoire atteinte (nombre l'éléments de données par mémoire)**

**Description** Le nombre d'éléments de données pouvant être sauvegardés est limité. Cette limite de mémoire a été atteinte. La connexion Ethernet a été fermée afin d'éviter la réception de données supplémentaires.

Le nombre maximum d'éléments pouvant être sauvegardés est défini dans le fichier XML contenant la configuration de connexion :

- Paragraphe <INTERNAL> ... </INTERNAL>
- Élément <BUFFERING ... Limit="..."/>
  - Nombre maximum possible : 512
- Si l'élément BUFFERING n'est pas configuré, la valeur par défaut est variable :
  - <BUFFERING Mode="FIFO" Limit="16"/>

**Fichier de configuration** Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.

Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Fichier	<b>Exemple</b> : ... \EXT.XML → EKI_INIT("EXT")

### Instruction de contrôle

- Dans le fichier XML contenant la configuration de connexion, vérifier quelle est la valeur limite du nombre d'éléments pouvant être sauvegardés.

**Solution: contrôler la mémoire de données de façon cyclique et la verrouiller avant que la limite soit atteinte**

### Description

La programmation peut permettre d'éviter que la limite de mémoire soit atteinte. Pour ce faire, on contrôle de façon cyclique le nombre d'éléments de données se trouvant actuellement dans la mémoire et verrouille la mémoire avant que la limite soit atteinte.

Les fonctions suivantes sont nécessaires :

- `EKI_STATUS = EKI_CheckBuffer(CHAR[], CHAR[])`  
Vérifie combien de données se trouvent encore dans la mémoire. La mémoire n'est pas modifiée.
- `EKI_STATUS = EKI_Lock(CHAR[])`  
Verrouille le traitement de données reçues. Les données ne peuvent plus être sauvegardées dans la mémoire.
- `EKI_STATUS = EKI_Unlock(CHAR[])`  
Déverrouille le traitement de données reçues. Les données sont à nouveau sauvegardées dans la mémoire.

`EKI_STATUS` est la variable de structure globale dans laquelle les valeurs renvoyées de la fonction sont écrites. L'élément `Buff` d'`EKI_STATUS` est important pour l'évaluation.

`Buff` contient la valeur suivante :

- Nombre d'éléments se trouvant encore dans la mémoire après un accès.

### Syntaxe

`GLOBAL STRUC EKI_STATUS INT Buff, Read, Msg_No, BOOL Connected, INT Counter`

### Exemple

```
...
DECL INT limit
DECL EKI_STATUS ret
...
limit = 16

ret = EKI_CheckBuffer("MyChannel", "MyData")

IF(ret.Buff >= limit - 1) THEN
 ret = EKI_Lock("MyChannel")
ELSE
 ret = EKI_Unlock("MyChannel")
ENDIF
```

### Cause: EKI : la mémoire EKI n'est pas lue

### Description

Le système externe écrit des données dans la mémoire EKI. Si ces données ne sont pas appelées par les instructions KRL, la mémoire EKI se remplit. Afin d'éviter la perte de données, la liaison est coupée lorsque la mémoire EKI est pleine. Le protocole TCP détecte la liaison coupée et interrompt l'échange de données. Le nombre maximum d'éléments de données a été écrit dans la mémoire EKI sans être lu.

Il est possible de vérifier si la mémoire EKI est lue de la façon suivante :

- Instruction de contrôle**
1. Sélectionner un programme et cliquer sur **Ouvrir**.
  2. Vérifier si l'instruction KRL **EKI\_Get** s'y trouve.

**Solution: Modifier le programme**

- Condition préalable**
- Groupe d'utilisateurs "Expert"
  - Mode T1, T2 ou AUT
- Procédure**
1. Marquer le programme dans le navigateur et appuyer sur **Ouvrir**. Le programme est affiché dans l'éditeur.
  2. Chercher l'endroit correspondant dans le programme et l'éditer.
  3. Fermer le fichier et répondre par **Oui** à la question de sécurité demandant si la modification doit être sauvegardée.

### 9.3.26 EKI01536

Code de message	■ EKI01536
Texte du message	■ Chaîne de caractères reçue trop longue
Type de message	■ Message de défaut
	
Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ <b>Cause:</b> chaîne de caractères envoyée par le système externe trop longue (&gt;&gt;&gt; Page 99)     <b>Solution:</b> adapter la programmation sur le système externe (&gt;&gt;&gt; Page 99)

**Cause: chaîne de caractères envoyée par le système externe trop longue**

**Description** Une chaîne de caractères envoyée par le système externe dépasse la longueur maximum utilisée. Un maximum de 3 600 caractères est autorisé.

- Instruction de contrôle**
- Contrôler la longueur des données envoyées par le système externe.

**Solution: adapter la programmation sur le système externe**

**Description** La programmation sur le système externe doit être adaptée de façon à ne plus envoyer de chaîne de caractères trop longue.

### 9.3.27 EKI01792

Code de message	■ EKI01792
Texte du message	■ La limite de la mémoire de réception est atteinte.
Type de message	■ Message de défaut
	

Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ <b>Cause:</b> Limite de mémoire atteinte (nombre d'octets de la mémoire totale) (&gt;&gt;&gt; Page 100)     <b>Solution:</b> augmenter la limite de mémoire dans le fichier XML (&gt;&gt;&gt; Page 100)

### Cause: Limite de mémoire atteinte (nombre d'octets de la mémoire totale)

<b>Description</b>	Le nombre d'octets pouvant être sauvegardés est limité. Cette limite de mémoire a été atteinte. La connexion Ethernet a été fermée afin d'éviter la réception de données supplémentaires.   Le nombre maximum d'octets pouvant être sauvegardés est défini dans le fichier XML contenant la configuration de connexion :       ■ Paragraphe &lt;INTERNAL&gt; ... &lt;/INTERNAL&gt;     ■ Élément &lt;BUFFSIZE Limit="..."/&gt;      ■ Nombre maximum possible : 65534           ■ Si l'élément BUFFSIZE n'est pas configuré, la valeur par défaut est valable :      ■ &lt;BUFFSIZE Limit="16384"/&gt;				
<b>Fichier de configuration</b>	Un fichier XML doit être configuré pour chaque connexion Ethernet. Le nom du fichier XML est simultanément la clé d'accès en KRL.  <table border="1"> <tr> <td>Répertoire</td><td>C:\KRC\ROBOTER\Config\User\Common\EthernetKRL</td></tr> <tr> <td>Fichier</td><td><b>Exemple :</b> ... \EXT.XML → EKI_INIT("EXT")</td></tr> </table>	Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL	Fichier	<b>Exemple :</b> ... \EXT.XML → EKI_INIT("EXT")
Répertoire	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL				
Fichier	<b>Exemple :</b> ... \EXT.XML → EKI_INIT("EXT")				
<b>Instruction de contrôle</b>	■ Dans le fichier XML contenant la configuration de connexion, vérifier quelle est la valeur limite du nombre d'octets.				

### Solution: augmenter la limite de mémoire dans le fichier XML

<b>Description</b>	La limite de mémoire peut être augmentée dans le fichier XML.
<b>Condition préalable</b>	■ Groupe d'utilisateurs "Expert"     ■ Mode T1, T2 ou AUT
<b>Procédure</b>	1. Modifier le fichier XML à votre convenance.     2. Si le fichier XML a été modifié hors ligne, le copier dans le répertoire prévu et écraser l'ancien fichier XML.     3. Redémarrer le contrôleur de robot avec les réglages <b>Dém. à froid</b> et <b>Nouvelle lecture des fichiers</b>.

#### 9.3.28 EKI02048

Code de message	■ EKI02048
Texte du message	■ La limite de temps du serveur est atteinte
Type de message	■ Message de défaut
	

Effet	■ ARRET D'URGENCE conforme à la trajectoire     ■ La saisie d'instructions actives (déplacements du robot, lancement du programme) est bloquée.
Cause(s) possible(s)	■ <b>Cause:</b> une erreur de logiciel sur le système externe entraîne un délai de temporisation (&gt;&gt;&gt; Page 101)     ■ <b>Solution:</b> éliminer le défaut dans le logiciel du système externe (&gt;&gt;&gt; Page 101)

### **Cause: une erreur de logiciel sur le système externe entraîne un délai de temporisation**

**Description** EKI est configurée en tant que serveur et attend la demande de connexion du système externe.

Une valeur pour <TIMEOUT Connect="..."/> est indiquée dans le fichier XML avec la configuration de connexion. Ce laps de temps s'est écoulé sans que le système externe ne se connecte au serveur. Ceci est causé par une erreur dans le logiciel du système externe.

**Instruction de contrôle** ■ Vérifier que le système externe est exempt d'erreurs de logiciel.

### **Solution: éliminer le défaut dans le logiciel du système externe**

**Description** Le défaut dans le logiciel du système externe doit être éliminé.



## 10 Annexe

### 10.1 Structure XML avancée pour les propriétés de connexion



La structure XML avancée ne doit être utilisée qu'après consultation de KUKA Roboter GmbH. (>>> 11 "SAV KUKA" Page 115)

#### Description

Des propriétés supplémentaires pour l'EKI peuvent être configurées dans le paragraphe <INTERNAL> ... </INTERNAL> du fichier de configuration :

Elément	Attribut	Description
TIMEOUT	Receive	Durée après laquelle la tentative de réception de données est interrompue (option) ■ 0 ... 65 534 ms Valeur par défaut : 0 ms
	Send	Durée après laquelle la tentative d'envoi de données est interrompue (option) ■ 0 ... 65 534 ms Valeur par défaut : 0 ms
BUFFSIZE	Receive	Taille du Socket utilisé lors de la réception de données (option) ■ 1 ... 65 534 octets Valeur par défaut : prédéfinie par le système
	Send	Taille du Socket utilisé lors de l'envoi de données (option) ■ 1 ... 65 534 octets Valeur par défaut : prédéfinie par le système

### 10.2 Augmentation de la mémoire



Une augmentation de la mémoire est seulement possible après consultation de KUKA Roboter GmbH. (>>> 11 "SAV KUKA" Page 115)

#### Description

Si la mémoire disponible est insuffisante, il est conseillé de vérifier le mode de programmation dans KRL et la configuration.

- Vérifier si une connexion est configurée de façon à ce que la mémoire soit entièrement occupée par les données reçues.
- Vérifier si plusieurs connexions sont définies et activées avec un flux élevé de données.

#### Condition préalable

- Groupe d'utilisateurs "Expert"
- Mode T1, T2 ou AUT

#### Procédure

1. Dans le répertoire C:\KRC\ROBOTER\Config\User\Common, ouvrir le fichier EthernetKRL.XML.
2. Inscrire la capacité de mémoire souhaitée en octets dans le paragraphe <EthernetKRL, dans l'élément <MemSize>.

```
<EthernetKRL>
 <Interface>
```

```
<MemSize>1048576</MemSize>
...
</EthernetKRL>
```

3. Fermer le fichier et répondre par **Oui** à la question de sécurité demandant si la modification doit être sauvegardée.
4. Redémarrer le contrôleur de robot avec les réglages **Dém. à froid** et **Nouvelle lecture des fichiers**.

### 10.3 Désactivation de l'émission de messages et du protocole de messages

#### Description

Dans les cas suivants, il est recommandé de désactiver l'émission automatique de messages sur la smartHMI .

- Des erreurs de durée de traitement apparaissent.
- EKI est utilisée dans l'interpréteur Submit.

Lorsque l'émission automatique de messages est désactivée, tous les messages de défaut continuent à être consignés par défaut. Si on souhaite ignorer sciemment des défauts ou des avertissements, p. ex. parce que le protocole des messages entraverait la performance du système et le ralentirait, ce mécanisme peut également être désactivé.

#### Condition préalable

- Groupe d'utilisateurs Expert

#### Procédure

1. Dans le répertoire C:\KRC\ROBOTER\Config\User\Common\EthernetKRL du contrôleur de robot, ouvrir le fichier de configuration de la connexion Ethernet.
2. Inscrire la ligne suivante dans le paragraphe <INTERNAL> ... </INTERNAL> du fichier XML :

```
<MESSAGES Logging="disabled" Display="disabled"/>
```

3. Sauvegarder la modification et fermer le fichier.



Lorsque l'émission automatique de messages est désactivée, la fonction EKI\_CHECK() peut être utilisée pour vérifier que les différentes instructions EKI sont exemptes de défauts.

### 10.4 Référence d'instruction

#### 10.4.1 Initialiser, ouvrir, fermer et effacer une connexion

RET = EKI_Init(CHAR[])	
Fonction	Initialise un canal pour la communication Ethernet   Les actions suivantes sont exécutées :       ■ Lecture de la configuration de connexion     ■ Réglage des mémoires de données     ■ Préparation de la connexion Ethernet
Paramètre	Type: CHAR   Nom du canal (nom du fichier XML contenant la configuration de connexion)
RET	Type: EKI_STATUS   Valeurs renvoyées de la fonction
Exemple	RET = EKI_Init("Channel_1")

<b>RET = EKI_Open(CHAR[])</b>	
Fonction	Ouvre un canal initialisé  Lorsqu'EKI est configurée en tant que client, l'EKI se connecte avec le système externe (= serveur).  Lorsque l'EKI est configurée en tant que serveur, l'EKI attend la demande de connexion du système externe (= client).
Paramètre	Type: CHAR Nom du canal
RET	Type: EKI_STATUS Nom du canal (nom du fichier XML contenant la configuration de connexion)
Exemple	RET = EKI_Open("Channel_1")

<b>RET = EKI_Close(CHAR[])</b>	
Fonction	Ferme un canal ouvert
Paramètre	Type: CHAR Nom du canal (nom du fichier XML contenant la configuration de connexion)
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_Close("Channel_1")

<b>RET = EKI_Clear(CHAR[])</b>	
Fonction	Efface un canal ainsi que toutes les mémoires de données correspondantes et ferme la connexion Ethernet.
Paramètre	Type: CHAR Nom du canal (nom du fichier XML contenant la configuration de connexion)
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_Clear("Channel_1")

Des informations concernant les valeurs renvoyées des différentes fonctions sont fournies ici : (>>> "Valeurs renvoyées" Page 42)

#### 10.4.2 Envoyer des données

<b>RET = EKI_Send(CHAR[], CHAR[], INT)</b>	
Fonction	Envoie des données par un canal  (>>> 6.2.4 "Envoi de données" Page 36)
Paramètre 1	Type: CHAR Nom du canal ouvert par lequel l'envoi doit être effectué

RET = EKI_Send(CHAR[], CHAR[], INT)	
Paramètre 2	Type: CHAR   Défini l'étendue des données à envoyer.   Pour une communication via structure XML :       ■ Impression de XPath de l'élément à envoyer de la structure XML configurée pour l'envoi de données.       Si seul l'élément racine est indiqué, l'ensemble de la structure XML est transféré (voir l'exemple 1). Si seule une partie de la structure XML doit être transférée, c'est-à-dire l'élément d'un niveau subordonné, il faut indiquer le chemin jusqu'à cet élément en partant de cet élément racine (voir l'exemple 2).       ■ Alternative : chaîne de caractères quelconque de longueur variable devant être transmise       Pour une communication via données brutes :       ■ chaîne de caractères quelconque devant être transmise :      ■ Pour des blocs de données binaires avec longueur fixe (attribut <code>Type= « BYTE »</code>) : chaîne de caractères quelconque de longueur fixe       La longueur de la chaîne de caractères en octets doit correspondre exactement à l'attribut configuré <code>Size</code>. En cas de dépassement, un message de défaut est émis, si la longueur n'est pas atteinte, un message d'avertissement est émis.    ■ Pour des blocs de données binaires avec séquence de suffixes variable (attribut <code>Type= « STREAM »</code>) : chaîne de caractères quelconque de longueur variable         La séquence de suffixes est envoyée automatiquement.   <b>Remarque :</b> si une chaîne de caractères quelconque de longueur variable contient un caractère ZÉRO ASCII, seule la partie concernant ce symbole est transférée. Si cela n'est pas souhaité, il faudra régler le paramètre 3 en conséquence.
Paramètre 3 (option)	Type: INT   N'est important que pour l'envoi de chaînes de caractères quelconques dont la longueur est variable (voir le paramètre 2) : nombre maximum de caractères à envoyer   Le caractère ZÉRO ASCII est ignoré. Si la chaîne de caractères est trop longue, celle-ci est coupée.
RET	Type: EKI_STATUS   Valeurs renvoyées de la fonction
Exemple 1	RET = EKI_Send("Channel_1", "Root")
Exemple 2	RET = EKI_Send("Channel_1", "Root/Test")
Exemple 3	RET = EKI_Send("Channel_1", MyBytes[])
Exemple 4	RET = EKI_Send("Channel_1", MyBytes[], 6)

Des informations concernant les valeurs renvoyées des différentes fonctions sont fournies ici : (>>> "Valeurs renvoyées" Page 42)

## 10.4.3 Écrire des données

<b>RET = EKI_SetBool(CHAR[], CHAR[], BOOL)</b>	
Fonction	Ecrit une valeur booléenne dans une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: BOOL Valeur écrite dans la mémoire
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_SetBool("Channel_1", "Root/Activ", true)

<b>RET = EKI_SetFrame(CHAR[], CHAR[], FRAME)</b>	
Fonction	Ecrit une valeur de type FRAME dans une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: FRAME Valeur écrite dans la mémoire
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET= EKI_SetFrame("Channel_1", "Root/BASE", {X 0.0, Y 0.0, Z 0.0, A 0.0, B 0.0, C 0.0})

<b>RET = EKI_SetInt(CHAR[], CHAR[], INT)</b>	
Fonction	Ecrit une valeur entière dans une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: INT Valeur écrite dans la mémoire
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_SetInt("Channel_1", "Root/List", 67234)

<b>RET = EKI_SetReal(CHAR[], CHAR[], REAL)</b>	
Fonction	Écrit une valeur à virgule flottante dans une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML

<b>RET = EKI_SetReal(CHAR[], CHAR[], REAL)</b>	
Paramètre 3	Type: REAL Valeur écrite dans la mémoire
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_SetReal("Channel_1", "Root/Number", 1.234)

<b>RET = EKI_SetString(CHAR[], CHAR[], CHAR[])</b>	
Fonction	Écrit une chaîne de caractères dans une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: CHAR Chaîne de caractères écrite dans la mémoire Nombre maximum de caractères : ■ <b>3 600</b>
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_SetString("Channel_1", "Root/Message", "Hello")

Des informations concernant les valeurs renvoyées des différentes fonctions sont fournies ici : (>>> "Valeurs renvoyées" Page 42)

#### 10.4.4 Lire des données

<b>RET = EKI_GetBool(CHAR[], CHAR[], BOOL)</b>	
Fonction	Lit une valeur booléenne d'une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: BOOL Valeur lue dans la mémoire
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_GetBool("Channel_1", "Root/Activ", MyBool)

<b>RET = EKI_GetBoolArray(Char[], Char[], Bool[])</b>	
Fonction	Lit une valeur booléenne d'une mémoire et copie la valeur dans le champ transmis par le programme KRL.   Les valeurs sont lues jusqu'à ce que le champ soit plein ou qu'il n'y ait plus aucun élément.
Paramètre 1	Type: CHAR   Nom du canal ouvert
Paramètre 2	Type: CHAR   Nom de la position dans la structure XML
Paramètre 3	Type: BOOL   Champ lu dans la mémoire   Nombre maximum d'éléments de champs lisibles :   ■ <b>512</b>
RET	Type: EKI_STATUS   Valeurs renvoyées de la fonction
Exemple	RET = EKI_GetBoolArray("Channel_1", "Root/Activ", MyBool[])

<b>RET = EKI_GetInt(Char[], Char[], Int)</b>	
Fonction	Lit une valeur entière d'une mémoire
Paramètre 1	Type: CHAR   Nom du canal ouvert
Paramètre 2	Type: CHAR   Nom de la position dans la structure XML
Paramètre 3	Type: INT   Valeur lue dans la mémoire
RET	Type: EKI_STATUS   Valeurs renvoyées de la fonction
Exemple	RET = EKI_GetInt("Channel_1", "Root/Numbers/One", MyInteger)

<b>RET = EKI_GetIntArray(Char[], Char[], Int[])</b>	
Fonction	Lit une valeur entière d'une mémoire et copie la valeur dans le champ transmis par le programme KRL.   Les valeurs sont lues jusqu'à ce que le champ soit plein ou qu'il n'y ait plus aucun élément.
Paramètre 1	Type: CHAR   Nom du canal ouvert
Paramètre 2	Type: CHAR   Nom de la position dans la structure XML
Paramètre 3	Type: INT   Champ lu dans la mémoire   Nombre maximum d'éléments de champs lisibles :   ■ <b>512</b>

**RET = EKI\_GetIntArray(CHAR[], CHAR[], INT[])**

RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_GetIntArray("Channel_1", "Root/Numbers/One", MyInteger[])

**RET = EKI\_GetReal(CHAR[], CHAR[], REAL)**

Fonction	Lit une valeur à virgule flottante d'une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: REAL Valeur lue dans la mémoire
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_GetReal("Channel_1", "Root/Position", MyReal)

**RET = EKI\_GetRealArray(CHAR[], CHAR[], REAL[])**

Fonction	Lit une valeur à virgule flottante d'une mémoire et copie la valeur dans le champ transmis par le programme KRL.  Les valeurs sont lues jusqu'à ce que le champ soit plein ou qu'il n'y ait plus aucun élément.
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: REAL Champ lu dans la mémoire Nombre maximum d'éléments de champs lisibles : ■ <b>512</b>
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_GetRealArray("Channel_1", "Root/Position", MyReal[])

**RET = EKI\_GetString(CHAR[], CHAR[], CHAR[])**

Fonction	Lit une chaîne de caractères d'une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML ou nom de l'élément dans les données brutes

<b>RET = EKI_GetString(CHAR[], CHAR[], CHAR[])</b>	
Paramètre 3	Type: CHAR Chaîne de caractères lue dans la mémoire Nombre maximum de caractères : ■ <b>3 600</b>
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple XML	RET = EKI_GetString("Channel_1", "Root/Message", MyChars[])
Exemple binaire	RET = EKI_GetString("Channel_1", "Streams", MyStream[])

<b>RET = EKI_GetFrame(CHAR[], CHAR[], FRAME)</b>	
Fonction	Lit une valeur de type FRAME d'une mémoire
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: FRAME Valeur lue dans la mémoire
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_GetFrame("Channel_1", "Root/TCP", MyFrame)

<b>RET = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])</b>	
Fonction	Lit une valeur de type FRAME d'une mémoire et copie la valeur dans le champ transmis par le programme KRL.  Les valeurs sont lues jusqu'à ce que le champ soit plein ou qu'il n'y ait plus aucun élément.
Paramètre 1	Type: CHAR Nom du canal ouvert
Paramètre 2	Type: CHAR Nom de la position dans la structure XML
Paramètre 3	Type: FRAME Champ lu dans la mémoire Nombre maximum d'éléments de champs lisibles : ■ <b>512</b>
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_GetFrameArray("Channel_1", "Root/TCP", MyFrame[])

Des informations concernant les valeurs renvoyées des différentes fonctions sont fournies ici : (>>> "Valeurs renvoyées" Page 42)

### 10.4.5 Vérifier que la fonction est exempte de défauts

<b>EKI_CHECK(EKI_STATUS, EKrIMsgType, CHAR[])</b>	
Fonction	Vérifie si un défaut s'est produit lors de l'exécution d'une fonction :      ■ Le numéro de défaut est lu et le message correspondant est émis sur la smartHMI. (voir le paramètre 1)     ■ Option : si le nom de canal est indiqué, il y aura interrogation de la présence de défauts lors de la réception de données (voir le paramètre 3).
Paramètre 1	EKI_STATUS Valeurs renvoyées de la fonction contrôlée
Paramètre 2	Type: ENUM Type de message émis sur la smartHMI :      ■ <b>#NOTIFY</b>: Message de remarque     ■ <b>#STATE</b>: Message d'état     ■ <b>#QUIT</b>: Message d'acquittement     ■ <b>#WAITING</b>: Message d'attente
Paramètre 3 (option)	Type: CHAR Nom du canal ouvert devant être contrôlé
Exemple 1	EKI_CHECK(RET,#QUIT)
Exemple 2	EKI_CHECK(RET,#NOTIFY,"MyChannelName")

Des informations concernant les valeurs renvoyées des différentes fonctions sont fournies ici : (>>> "Valeurs renvoyées" Page 42)

### 10.4.6 Effacer, verrouiller, déverrouiller et contrôler la mémoire de données

<b>RET = EKI_Clear(CHAR[])</b>	
Fonction	Efface un canal ainsi que toutes les mémoires de données correspondantes et ferme la connexion Ethernet.
Paramètre	Type: CHAR Nom du canal (nom du fichier XML contenant la configuration de connexion)
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple	RET = EKI_Clear("Channel_1")

<b>RET = EKI_ClearBuffer(CHAR[], CHAR[])</b>	
Fonction	Efface la mémoire de données définie sans couper la connexion Ethernet.  (>>> 6.2.6 "Suppression de mémoires de données" Page 41)
Paramètre 1	Type: CHAR Nom du canal

<b>RET = EKI_ClearBuffer(CHAR[], CHAR[])</b>	
Paramètre 2	Type: CHAR Définit les mémoires de données à effacer. Pour une communication via données brutes :      ■ Nom des données brutes à effacer de la structure XML configurée pour la réception de données.      Pour une communication via structure XML :      ■ Impression XPATH de l'élément de la structure XML configurée pour la réception de données ou de la structure XML pour l'envoi de données dont la mémoire de données doit être effacée.
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple 1	RET = EKI_ClearBuffer("Channel_1", "RawData")
Exemple 2	RET = EKI_ClearBuffer("Channel_1", "Ext/Activ/Flag")

<b>RET = EKI_Lock(CHAR[])</b>	
Fonction	Verrouille le traitement de données reçues. Les données ne peuvent plus être sauvegardées dans la mémoire.
Paramètre	Type: CHAR Nom du canal
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction

<b>RET = EKI_Unlock(CHAR[])</b>	
Fonction	Déverrouille le traitement de données reçues. Les données sont à nouveau sauvegardées dans la mémoire.
Paramètre	Type: CHAR Nom du canal
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction

<b>RET = EKI_CheckBuffer(CHAR[], CHAR[])</b>	
Fonction	Vérifie combien de données se trouvent encore dans la mémoire. La mémoire n'est pas modifiée. De plus, l'horodatage de l'élément de données étant le prochain à être prélevé dans la mémoire est renvoyé.
Paramètre 1	Type: CHAR Nom du canal

RET = EKI_CheckBuffer(CHAR[], CHAR[])	
Paramètre 2	Type: CHAR Définit la mémoire de données à contrôler. Pour une communication via données brutes :      ■ Nom des données brutes à contrôler de la structure XML configurée pour la réception de données.      Pour une communication via structure XML :      ■ Impression XPATH de l'élément de la structure XML configurée pour la réception de données ou de la structure XML pour l'envoi de données dont la mémoire de données doit être contrôlée.
RET	Type: EKI_STATUS Valeurs renvoyées de la fonction
Exemple 1	RET = EKI_CheckBuffer("Channel_1", "RawData")
Exemple 2	RET = EKI_CheckBuffer("Channel_1", "Ext/Activ/Flag")

Des informations concernant les valeurs renvoyées des différentes fonctions sont fournies ici : (>>> "Valeurs renvoyées" Page 42)

## 11 SAV KUKA

### 11.1 Demande d'assistance

**Introduction** Cette documentation comprenant des informations relatives au service et à la commande vous fera office d'aide lors de l'élimination de défauts. La filiale locale est à votre disposition pour toute autre demande.

**Informations** **Pour traiter toute demande SAV, nous nécessitons les informations suivantes :**

- Description du problème et indications concernant la durée et la fréquence du défaut
- Informations les plus complètes possibles concernant les composants matériels et logiciels de l'ensemble du système

La liste suivante fournit des repères quant aux informations importantes dans la plupart des cas :

- Type et numéro de série de la cinématique, p. ex. du manipulateur
- Type et numéro de série de la commande
- Type et numéro de série de l'alimentation en énergie
- Désignation et version du logiciel System Software
- Désignations et versions de composants logiciels supplémentaires / différents ou de modifications
- Pack de diagnostic KRCDiag

En supplément pour KUKA Sunrise : projets existants, applications comprises

Pour des versions de KUKA System Software antérieures à V8 : archives du logiciel (KRCDiag n'est pas encore disponible ici.)

- Application existante
- Axes supplémentaires existants

### 11.2 Assistance client KUKA

**Disponibilité** Notre assistance client KUKA est disponible dans de nombreux pays. Nous sommes à votre disposition pour toute question.

**Argentine** Ruben Costantini S.A. (agence)  
Luis Angel Huergo 13 20  
Parque Industrial  
2400 San Francisco (CBA)  
Argentine  
Tél. +54 3564 421033  
Fax +54 3564 428877  
ventas@costantini-sa.com

**Australie** KUKA Robotics Australia Pty Ltd  
45 Fennell Street  
Port Melbourne VIC 3207  
Australie  
Tél. +61 3 9939 9656  
info@kuka-robotics.com.au  
www.kuka-robotics.com.au

<b>Belgique</b>	KUKA Automatisering + Robots N.V.  Centrum Zuid 1031  3530 Houthalen  Belgique  Tél. +32 11 516160  Fax +32 11 526794  <a href="mailto:info@kuka.be">info@kuka.be</a>  <a href="http://www.kuka.be">www.kuka.be</a>
<b>Brésil</b>	KUKA Roboter do Brasil Ltda.  Travessa Claudio Armando, nº 171  Bloco 5 - Galpões 51/52  Bairro Assunção  CEP 09861-7630 São Bernardo do Campo - SP  Brésil  Tél. +55 11 4942-8299  Fax +55 11 2201-7883  <a href="mailto:info@kuka-roboter.com.br">info@kuka-roboter.com.br</a>  <a href="http://www.kuka-roboter.com.br">www.kuka-roboter.com.br</a>
<b>Chili</b>	Robotec S.A. (agence)  Santiago de Chile  Chili  Tél. +56 2 331-5951  Fax +56 2 331-5952  <a href="mailto:robotec@robotec.cl">robotec@robotec.cl</a>  <a href="http://www.robotec.cl">www.robotec.cl</a>
<b>Chine</b>	KUKA Robotics Chine Co., Ltd.  No. 889 Kungang Road  Xiaokunshan Town  Songjiang District  201614 Shanghai  P. R. de Chine  Tél. +86 21 5707 2688  Fax +86 21 5707 2603  <a href="mailto:info@kuka-robotics.cn">info@kuka-robotics.cn</a>  <a href="http://www.kuka-robotics.com">www.kuka-robotics.com</a>
<b>Allemagne</b>	KUKA Roboter GmbH  Zugspitzstr. 140  86165 Augsburg  Allemagne  Tél. +49 821 797-1926  Fax +49 821 797-41 1926  <a href="mailto:Hotline.robotics.de@kuka.com">Hotline.robotics.de@kuka.com</a>  <a href="http://www.kuka-roboter.de">www.kuka-roboter.de</a>

**France** KUKA Automatismes + Robotique SAS  
Techvallée  
6, Avenue du Parc  
91140 Villebon S/Yvette  
France  
Tél. +33 1 6931660-0  
Fax +33 1 6931660-1  
commercial@kuka.fr  
www.kuka.fr

**Inde** KUKA Robotics India Pvt. Ltd.  
Office Number-7, German Centre,  
Level 12, Building No. - 9B  
DLF Cyber City Phase III  
122 002 Gurgaon  
Haryana  
Inde  
Tél. +91 124 4635774  
Fax +91 124 4635773  
info@kuka.in  
www.kuka.in

**Italie** KUKA Roboter Italia S.p.A.  
Via Pavia 9/a - int.6  
10098 Rivoli (TO)  
Italie  
Tél. +39 011 959-5013  
Fax +39 011 959-5141  
kuka@kuka.it  
www.kuka.it

**Japon** KUKA Robotics Japan K.K.  
YBP Technical Center  
134 Godo-cho, Hodogaya-ku  
Yokohama, Kanagawa  
240 0005  
Japon  
Tél. +81 45 744 7691  
Fax +81 45 744 7696  
info@kuka.co.jp

**Canada** KUKA Robotics Canada Ltd.  
6710 Maritz Drive - Unit 4  
Mississauga  
L5W 0A1  
Ontario  
Canada  
Tél. +1 905 670-8600  
Fax +1 905 670-8604  
info@kukarobotics.com  
www.kuka-robotics.com/canada

<b>Corée</b>	KUKA Robotics Korea Co. Ltd.  RIT Center 306, Gyeonggi Technopark  1271-11 Sa 3-dong, Sangnok-gu  Ansan City, Gyeonggi Do  426-901  Corée  Tél. +82 31 501-1451  Fax +82 31 501-1461  <a href="mailto:info@kukakorea.com">info@kukakorea.com</a>
<b>Malaisie</b>	KUKA Robot Automation (M) Sdn Bhd  South East Asia Regional Office  No. 7, Jalan TPP 6/6  Taman Perindustrian Puchong  47100 Puchong  Selangor  Malaisie  Tél. +60 (03) 8063-1792  Fax +60 (03) 8060-7386  <a href="mailto:info@kuka.com.my">info@kuka.com.my</a>
<b>Mexique</b>	KUKA de México S. de R.L. de C.V.  Progreso #8  Col. Centro Industrial Puente de Vigas  Tlalnepantla de Baz  54020 Estado de México  Mexique  Tél. +52 55 5203-8407  Fax +52 55 5203-8148  <a href="mailto:info@kuka.com.mx">info@kuka.com.mx</a>  <a href="http://www.kuka-robotics.com/mexico">www.kuka-robotics.com/mexico</a>
<b>Norvège</b>	KUKA Sveiseanlegg + Roboter  Sentrumsvegen 5  2867 Hov  Norvège  Tél. +47 61 18 91 30  Fax +47 61 18 62 00  <a href="mailto:info@kuka.no">info@kuka.no</a>
<b>Autriche</b>	KUKA Roboter CEE GmbH  Gruberstraße 2-4  4020 Linz  Autriche  Tél. +43 7 32 78 47 52  Fax +43 7 32 79 38 80  <a href="mailto:office@kuka-roboter.at">office@kuka-roboter.at</a>  <a href="http://www.kuka.at">www.kuka.at</a>

**Pologne**  
KUKA Roboter CEE GmbH Poland  
Spółka z ograniczoną odpowiedzialnością  
Oddział w Polsce  
Ul. Porcelanowa 10  
40-246 Katowice  
Pologne  
Tél. +48 327 30 32 13 or -14  
Fax +48 327 30 32 26  
ServicePL@kuka-roboter.de

**Portugal**  
KUKA Robots IBÉRICA, S.A.  
Rua do Alto da Guerra n° 50  
Armazém 04  
2910 011 Setúbal  
Portugal  
Tél. +351 265 729 780  
Fax +351 265 729 782  
info.portugal@kukapt.com  
www.kuka.com

**Russie**  
KUKA Robotics RUS  
Werbnaja ul. 8A  
107143 Moskau  
Russie  
Tél. +7 495 781-31-20  
Fax +7 495 781-31-19  
info@kuka-robotics.ru  
www.kuka-robotics.ru

**Suède**  
KUKA Svetsanläggningar + Robotar AB  
A. Odhners gata 15  
421 30 Västra Frölunda  
Suède  
Tél. +46 31 7266-200  
Fax +46 31 7266-201  
info@kuka.se

**Suisse**  
KUKA Roboter Schweiz AG  
Industriestr. 9  
5432 Neuenhof  
Suisse  
Tél. +41 44 74490-90  
Fax +41 44 74490-91  
info@kuka-roboter.ch  
www.kuka-roboter.ch

<b>Espagne</b>	KUKA Robots IBÉRICA, S.A. Pol. Industrial Torrent de la Pastera Carrer del Bages s/n 08800 Vilanova i la Geltrú (Barcelona) Espagne Tél. +34 93 8142-353 Fax +34 93 8142-950 comercial@kukarob.es www.kuka.es
<b>Afrique du Sud</b>	Jendamark Automation LTD (agence)) 76a York Road North End 6000 Port Elizabeth Afrique du Sud Tél. +27 41 391 4700 Fax +27 41 373 3869 www.jendamark.co.za
<b>Taiwan</b>	KUKA Robot Automation Taiwan Co., Ltd. No. 249 Pujong Road Jungli City, Taoyuan County 320 Taïwan, R. O. C. Tél. +886 3 4331988 Fax +886 3 4331948 info@kuka.com.tw www.kuka.com.tw
<b>Thaïlande</b>	KUKA Robot Automation (M) Sdn Bhd Thailand Office c/o Maccall System Co. Ltd. 49/9-10 Soi Kingkaew 30 Kingkaew Road Tt. Rachatheva, A. Bangpli Samutprakarn 10540 Thaïlande Tél. +66 2 7502737 Fax +66 2 6612355 atika@ji-net.com www.kuka-roboter.de
<b>Tchèque</b>	KUKA Roboter Austria GmbH Organisation Tschechien und Slowakei Sezemická 2757/2 193 00 Praha Horní Počernice République tchèque Tél. +420 22 62 12 27 2 Fax +420 22 62 12 27 0 support@kuka.cz

**Hongrie** KUKA Robotics Hungaria Kft.  
Fő út 140  
2335 Taksony  
Hongrie  
Tél. +36 24 501609  
Fax +36 24 477031  
info@kuka-robotics.hu

**Etats-Unis** KUKA Robotics Corporation  
51870 Shelby Parkway  
Shelby Township  
48315-1787  
Michigan  
Etats-Unis  
Tél. +1 866 873-5852  
Fax +1 866 329-5852  
info@kukarobotics.com  
www.kukarobotics.com

**Royaume-Uni** KUKA Robotics UK Ltd  
Great Western Street  
Wednesbury West Midlands  
WS10 7LL  
Royaume-Uni  
Tél. +44 121 505 9970  
Fax +44 121 505 6589  
service@kuka-robotics.co.uk  
www.kuka-robotics.co.uk



## Index

### A

Annexe 103  
Aperçu, Ethernet KRL 11  
Aperçu, fonctions 32  
Assistance client KUKA 115  
Astuces de programmation 33

### C

CAST\_FROM() 39, 51  
CAST\_TO() 37, 51  
Cible 7  
Communication 11  
Conditions requises par le système 19  
Configuration 23  
Configuration, connexion Ethernet 11, 25  
Configurations à titre d'exemple 47  
Connaissances requises 7  
Connexion Ethernet, configuration 11, 25  
Connexion réseau 23  
Connexion, surveillance 12

### D

Demande d'assistance 115  
Description du produit 11  
Diagnostic 59  
Documentation, robot industriel 7  
Données de diagnostic, affichage 59  
Défragmentation 44  
Désinstallation, Ethernet KRL 19, 21

### E

EKI 8  
EKI\_CHECK() 42, 44, 112  
EKI\_CheckBuffer() 34, 113  
EKI\_Clear() 41, 105, 112  
EKI\_ClearBuffer() 41, 112  
EKI\_Close() 36, 105  
EKI\_GetBool() 108  
EKI\_GetBoolArray() 109  
EKI\_GetFrame() 111  
EKI\_GetFrameArray() 111  
EKI\_GetInt() 109  
EKI\_GetIntArray() 109  
EKI\_GetReal() 110  
EKI\_GetRealArray() 110  
EKI\_GetString() 39, 110  
EKI\_Init() 34  
EKI\_Lock() 113  
EKI\_Open() 36, 68, 69, 105  
EKI\_Send() 36, 78, 80, 84, 105  
EKI\_SetBool() 107  
EKI\_SetFrame() 107  
EKI\_SetInt() 107  
EKI\_SetReal() 107  
EKI\_SetString() 108  
EKI\_STATUS 42  
EKI\_Unlock() 113  
EOS 8

### Ethernet 8

Ethernet KRL, aperçu 11  
Ethernet, interfaces 23  
EthernetKRL\_Server.exe 47  
Exemples, intégration 47

### F

FIFO 8, 13  
Flux de données 8  
Fonctions 11  
Fonctions, aperçu 32  
Formations 7  
Fragmentation 44

### H

Horodatage 42

### I

Installation 19  
Installation avec smartHMI 20  
Installation avec WorkVisual 19  
Introduction 7  
IP 9

### K

KLI 8, 23  
KR C 8  
KRL 8

### L

Licences 9  
LIFO 8, 13, 44  
Logiciels 19

### M

Marques 9  
Matériel 19  
Messages 61  
Messages d'événements 15, 43  
Messages, désactivation 104  
Mise à jour, Ethernet KRL 19, 20  
Mode client 14, 36  
Mode serveur 14, 36  
Moniteur de diagnostic (option de menu) 59  
Mémoire, augmentation 103

### O

Open-Source 9

### P

Perte de connexion 11  
Ping 12  
Programmation 25  
Programme de serveur 47  
Programme de serveur, interface utilisateur 48  
Programme de serveur, réglage des paramètres 49  
Programmes à titre d'exemple 47

Propriétés 11  
Protocole de défauts 61

**R**

Remarques 7  
Remarques relatives à la sécurité 7  
Réactions aux défauts, programmation 44  
Référence d'instruction 104

**S**

Sauvegarde de données 13  
SAV, KUKA Roboter GmbH 115  
smarHMI 8  
Socket 8  
Surveillance, connexion 12  
Sécurité 17  
Séquence de suffixes 8

**T**

Table de messages 61  
TCP/IP 9  
Termes utilisés 8  
Termes, utilisés 8  
Traitement de défauts 15  
Types de protocole 14

**U**

UDP/IP 9  
Utilisation conforme aux fins prévues 16  
Utilisation, conforme aux fins prévues 16

**X**

XML 9  
XPath 9, 29, 31

**Z**

Échange de données 12

